Representation Learning and Dimensionality Reduction using Autoencoders and PCA for Handwritten Digit Classification

Programming Assignment IV

CS601T: Deep Learning

Group 30

Submitted by

Anushaa B (Roll No.: CS24BT032)
Skanda P (Roll No.: CS24BT045)
Thiaana B (Roll No.: CS24BT031)

Under the Supervision of

Prof. Dileep A. D.
Professor, Department of Computer Science and Engineering



Department of Computer Science and Engineering
Indian Institute of Technology Dharwad

October 2026

Abstract

This assignment explores dimensionality reduction and feature learning for handwritten digit classification using a subset of the MNIST dataset containing five classes. Principal Component Analysis (PCA) is used to reduce the dimensionality of images from 784 to 32, 64, 128, and 256 dimensions, followed by classification using fully connected neural networks (FCNNs).

Autoencoders with different architectures and bottleneck dimensions are implemented to reconstruct images and extract compressed representations for classification. Their reconstruction and classification performances are evaluated and compared with PCA-based methods and results from the previous assignment. Denoising autoencoders are further studied using different noise levels to assess their reconstruction capabilities and classification performance. Finally, learned weights are visualized to examine the features captured by standard and denoising autoencoders.

The experiments provide a comparative analysis of dimensionality reduction and feature learning techniques for image reconstruction and classification. Reconstruction error decreases steadily as the bottleneck grows (test error 9.97×10^{-3} at 32 dimensions to 9.6×10^{-4} at 256 for the one-hidden-layer autoencoder), whereas classification accuracy does not improve with dimension: every representation performs best or near-best at 32 dimensions, where the test accuracies of PCA (98.47%), the one-hidden-layer autoencoder (98.50%), the three-hidden-layer autoencoder (98.55%) and the denoising autoencoders (98.66%) all match or exceed the 98.02% obtained on raw pixels in Assignment 3 with a 24-fold smaller input. The differences among the 32-dimensional representations are only a few test images and therefore small relative to sampling variability; we report them with that caveat. Weight visualization shows that denoising autoencoders learn smoother, lower-magnitude, more stroke-like filters than the standard autoencoder.

Contents

Abstract **i**

1 Introduction **1**

1.1 Motivation for Representation Learning 1
1.2 The Curse of Dimensionality in Image Processing 1
1.3 Linear versus Nonlinear Dimensionality Reduction 1
1.4 Autoencoder Paradigms: Undercomplete, Deep, and Denoising 2
1.5 Objectives of Programming Assignment 4 2
1.6 Group 30 Dataset Specifications 2
1.7 Organization of the Report 2

2 Proposed Methodology **3**

2.1 Problem Formulation and Representation Learning 3
2.2 Principal Component Analysis (PCA) 3
2.2.1 Mean-Centering and Covariance Computation 4
2.2.2 Eigendecomposition and Direction of Projection 4
2.2.3 Dimension Reduction and Projection 5
2.2.4 Reconstruction and Cumulative Retained Variance 5
2.3 Autoencoders (Auto-Associative Neural Networks) 5
2.3.1 Architectural Components: Encoder and Decoder 6
2.3.2 Symmetric Multi-Hidden-Layer Autoencoders 6
2.3.3 Objective Function and Reconstruction Error 7
2.4 Theoretical Link between PCA and Autoencoders 7
2.5 Denoising Autoencoders (DAE) 8
2.5.1 Stochastic Corruption Process 8
2.5.2 Denoising Objective and Manifold Learning 9
2.6 Weight Visualization and Receptive Fields 9
2.6.1 Mathematical Derivation of Maximal Activation Stimulus 9
2.6.2 Quantitative Filter Sharpness via Laplacian Variance 10

2.7 Downstream Classification using Fully Connected Neural Networks . . 10
2.7.1 Network Architecture and Forward Propagation 10
2.7.2 Cross-Entropy Objective and Gradient Descent Optimization . . 11

3 Experimental Setup 12
3.1 Dataset Description and Preprocessing 12
3.2 PCA Configuration 13
3.3 Autoencoder Architectures and Training Hyperparameters 14
3.4 Denoising Autoencoder Configuration 15
3.5 FCNN Classifier Architectures and Training Setup 15
3.6 Evaluation Metrics 16

4 Task 1: Dimension Reduction using PCA 18
4.1 Variance Retained by Principal Components 18
4.2 Validation Accuracy across FCNN Architectures 19
4.3 Test Performance of the Selected Architectures 19
4.3.1 Convergence History and Confusion Matrices 19
4.3.2 Detailed Error Analysis on Test Predictions 20
4.4 Observation on the Best Reduced Dimension 21
4.5 Comparison with Assignment 3 Best Result 21

5 Task 2: Autoencoders for Reconstructing Images 25
5.1 One-Hidden-Layer Autoencoders (AE-1) 25
5.1.1 Visual Reconstruction Analysis for AE-1 25
5.2 Three-Hidden-Layer Autoencoders (AE-3) 27
5.2.1 Visual Reconstruction Analysis for AE-3 27
5.3 Comparison and Architectural Trade-Offs: AE-1 versus AE-3 27
5.3.1 Analytical Discussion of Findings 28

6 Task 3: Classification using 1-Hidden-Layer Autoencoder Representations 30
6.1 Validation Accuracy across FCNN Architectures 30
6.2 Test Performance of Selected Architectures 30
6.2.1 Loss Trajectories and Confusion Matrices 31
6.2.2 Error Analysis of Test Predictions 31
6.3 Best Reduced Dimension Representation 32
6.4 Comparison with Assignment 3 and Task 1 (PCA) 32

7 Task 4: Classification using 3-Hidden-Layer Autoencoder Representations 36
7.1 Validation Accuracy across FCNN Architectures 36
7.2 Test Performance of Selected Architectures 36
7.2.1 Loss Convergence Trajectories and Confusion Matrices 37
7.2.2 Error Analysis of Test Predictions 37
7.3 Best Reduced Dimension Representation 38
7.4 Comprehensive Comparison across All Paradigms 38

8 Task 5: Denoising Autoencoders for Reconstruction and Classification 43
8.1 Selection of Bottleneck Dimension and Noise Model 43
8.2 Average Reconstruction Error on Clean and Noisy Inputs 44
8.3 Visual Reconstruction Analysis for DAE 44
8.4 Classification Performance using DAE Representations 45
8.4.1 Loss Trajectories and Confusion Matrices 46
8.4.2 Detailed Error Analysis and Confusion Matrix Inspection 46
8.5 Theoretical Analysis: Why Denoising Enhances Classification 46

9 Task 6: Weight Visualization and Filter Analysis 50
9.1 Encoder Weights as Maximal Activation Images 50
9.1.1 Standard Autoencoder Weight Receptive Fields (0% Noise) . . . 50
9.1.2 Denoising Autoencoder Weight Receptive Fields (20% and 40% Noise) 51
9.2 Quantitative Statistical Comparison of Learned Weights 52
9.3 Comparative Analysis: Standard versus Denoising Weights 52

10 Conclusion and Comparative Discussion 55
10.1 Master Summary of Experimental Results 55
10.2 Fundamental Scientific Insights 55

References 58

List of Figures

3.1 Representative sample handwritten digits from the Group 30 training dataset across the five classes ($\{3, 4, 5, 7, 8\}$). 13

4.1 Validation accuracy heatmap for the nine FCNN architectures across the four PCA reduced dimensions. 20
4.2 Training loss trajectory and test confusion matrix for PCA-32 using `arch_3l_v3` ($[256, 256, 256]$). 21
4.3 Training loss trajectory and test confusion matrix for PCA-64 using `arch_3l_v3` ($[256, 256, 256]$). 22
4.4 Training loss trajectory and test confusion matrix for PCA-128 using `arch_4l_v2` ($[256, 128, 64, 32]$). 23
4.5 Training loss trajectory and test confusion matrix for PCA-256 using `arch_5l_v2` ($[256, 256, 128, 128, 64]$). 23
4.6 Test accuracy versus PCA reduced dimension, compared against the Assignment 3 raw pixel baseline (98.02%). 24

5.1 Reconstructions on test images for 1-hidden-layer autoencoders (AE-1) across bottleneck dimensions. 26
5.2 Reconstruction fidelity on training and validation sets for 1-hidden-layer autoencoder with bottleneck $l = 32$. 26
5.3 Reconstructions on test images for 3-hidden-layer deep autoencoders (AE-3) across bottleneck dimensions. 28
5.4 Reconstruction fidelity on training and validation sets for 3-hidden-layer autoencoder with bottleneck $l = 32$. 28
5.5 Reconstruction MSE versus bottleneck dimension for AE-1 and AE-3 on linear (left) and logarithmic (right) scales. 29

6.1 Validation accuracy heatmap for FCNN classifiers trained on 1-hidden-layer autoencoder representations. 32
6.2 Training loss curve and test confusion matrix for AE1-32 using `arch_3l_v1` ($[512, 256, 128]$). 33

6.3 Training loss curve and test confusion matrix for AE1-64 using `arch_4l_v3` ([384, 256, 128, 64]). 34
6.4 Training loss curve and test confusion matrix for AE1-128 using `arch_3l_v2` ([256, 128, 64]). 34
6.5 Training loss curve and test confusion matrix for AE1-256 using `arch_5l_v1` ([512, 384, 256, 128, 64]). 35
6.6 Test accuracy across 1-hidden-layer autoencoder bottleneck dimensions compared to Assignment 3. 35

7.1 Validation accuracy heatmap for FCNN classifiers trained on 3-hidden-layer autoencoder representations. 38
7.2 Training loss curve and test confusion matrix for AE3-32 using `arch_4l_v1` ([512, 256, 128, 64]). 39
7.3 Training loss curve and test confusion matrix for AE3-64 using `arch_4l_v3` ([384, 256, 128, 64]). 40
7.4 Training loss curve and test confusion matrix for AE3-128 using `arch_4l_v3` ([384, 256, 128, 64]). 41
7.5 Training loss curve and test confusion matrix for AE3-256 using `arch_4l_v3` ([384, 256, 128, 64]). 41
7.6 Test classification accuracy versus bottleneck dimension for 3-hidden-layer autoencoder representations. 42

8.1 Reconstructions produced by the 20% Denoising Autoencoder ($l = 32$) on training, validation, and test splits. 45
8.2 Reconstructions produced by the 40% Denoising Autoencoder ($l = 32$) on training, validation, and test splits. 45
8.3 Classifier training loss convergence and test confusion matrices on 32-dimensional DAE representations. 48
8.4 Test accuracy comparison between Task 3 plain autoencoder codes and Task 5 denoising autoencoder codes (20% and 40% noise). 49

9.1 Receptive field filters (28×28 weight maps) for all 32 bottleneck neurons in the standard 1-hidden-layer autoencoder (0% noise). 51
9.2 Receptive field filters for all 32 bottleneck neurons in the Denoising Autoencoder trained with 20% noise. 52
9.3 Receptive field filters for all 32 bottleneck neurons in the Denoising Autoencoder trained with 40% noise. 53

9.4 Statistical comparison of encoder weights: weight value distributions (left) and Laplacian variance sharpness metrics (right). 54
9.5 Direct side-by-side receptive field comparison across corresponding bottleneck neurons for Standard AE (top), DAE 20% (middle), and DAE 40% (bottom). 54

10.1 Comprehensive comparison of test classification accuracy across all dimensionality reduction techniques, evaluated against the Assignment 3 raw pixel baseline (98.02%). 57

List of Tables

3.1 Distribution of samples per class across dataset splits for Group 30. . . 12
3.2 Trainable parameter counts for 1-hidden-layer (AE-1) and 3-hidden-layer (AE-3) autoencoders across bottleneck dimensions. 14
3.3 FCNN classifier architecture specifications and total parameter counts across input dimensions. 16

4.1 Cumulative variance retained by PCA on the Group 30 training dataset. 18
4.2 Validation classification accuracy (%) across nine FCNN architectures for each PCA dimension. Bold entries indicate the selected optimal architecture for each dimension. 19
4.3 Performance on the test dataset for the best FCNN architectures selected per PCA dimension. 20
4.4 Comparison of PCA classification performance against the best result of Assignment 3. 22

5.1 Reconstruction performance (MSE) and training convergence for the 1-hidden-layer autoencoders (AE-1). 25
5.2 Reconstruction performance (MSE) and training convergence for the 3-hidden-layer autoencoders (AE-3). 27
5.3 Direct comparison of test reconstruction MSE between 1-hidden-layer (AE-1) and 3-hidden-layer (AE-3) autoencoders. 27

6.1 Validation classification accuracy (%) across nine FCNN architectures using 1-hidden-layer autoencoder (AE-1) codes. Bold entries designate the optimal architecture per dimension. 31
6.2 Test set performance for the best FCNN architectures selected on 1-hidden-layer autoencoder representations. 31
6.3 Comparative test classification accuracy (%) between Assignment 3, Task 1 (PCA), and Task 3 (AE-1). 33

7.1 Validation classification accuracy (%) across nine FCNN architectures using 3-hidden-layer autoencoder (AE-3) codes. Bold entries indicate the best-performing architecture per dimension. 37
7.2 Test set performance for the best FCNN architectures trained on 3-hidden-layer autoencoder representations. 37
7.3 Four-way comparative test classification accuracy (%) across Assignment 3 baseline, Task 1 (PCA), Task 3 (AE-1), and Task 4 (AE-3). . . 39

8.1 Average reconstruction error (MSE) across dataset splits for plain and denoising autoencoders ($l = 32$). 44
8.2 Downstream FCNN classification performance on 32-dimensional latent representations using `arch_3l_v1` ($[512, 256, 128]$). 46

9.1 Quantitative statistical comparison of encoder weight matrices across standard and denoising autoencoders. 52

10.1 Master comparison of classification performance; architecture codes are defined in Table 3.3. 56

Chapter 1

Introduction

1.1 Motivation for Representation Learning

Handwritten-digit images contain correlated pixels and redundant background information. Compact representations can reduce this redundancy and provide useful features for reconstruction and classification.

1.2 The Curse of Dimensionality in Image Processing

Each 28×28 grayscale image is represented as a vector $\mathbf{x} \in \mathbb{R}^{784}$. Neighboring pixels are correlated, and many background pixels provide little class information. Reducing the input dimension can limit redundancy and simplify downstream classification.

1.3 Linear versus Nonlinear Dimensionality Reduction

This study compares two approaches:

- **PCA:** Projects mean-centered images onto directions of highest variance, producing a linear low-dimensional representation.
- **Autoencoders:** Learn compressed representations by reconstructing their inputs; deeper architectures can model nonlinear features.

1.4 Autoencoder Paradigms: Undercomplete, Deep, and Denoising

We evaluate shallow and deep autoencoders with bottleneck dimensions $l \in \{32, 64, 128, 256\}$, along with denoising autoencoders trained to reconstruct clean images from noisy inputs. Architectural and training details are provided in Chapter 2.

1.5 Objectives of Programming Assignment 4

The six tasks cover:

- **Task 1:** PCA-based dimensionality reduction and classification.
- **Task 2:** Image reconstruction with shallow and deep autoencoders.
- **Tasks 3–4:** Classification using autoencoder representations.
- **Task 5:** Denoising autoencoder reconstruction and classification.
- **Task 6:** Visualization and analysis of learned weights.

1.6 Group 30 Dataset Specifications

The experiments use Group 30's MNIST subset of digits $\{3, 4, 5, 7, 8\}$. Split sizes and preprocessing details are given in Chapter 3.

1.7 Organization of the Report

Chapter 2 describes the methods, Chapter 3 details the experimental setup, Chapters 4 through 9 present the results, and Chapter 10 summarizes the findings.

Chapter 2

Proposed Methodology

This chapter details the mathematical framework, foundational theory, formal equations, and algorithmic procedures governing all models evaluated in this work, adhering strictly to the formal notations established in the lecture curriculum.

2.1 Problem Formulation and Representation Learning

Let the training dataset be denoted by:

$$\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^{N}, \quad \mathbf{x}_n \in \mathbb{R}^d, \quad \mathbf{y}_n \in \mathbb{R}^d \tag{2.1}$$

where N is the total number of training exemplars ($N = 11,385$), d is the dimensionality of the flattened input vector ($d = 28 \times 28 = 784$), and $\mathbf{x}_n = [x_{n1}, x_{n2}, \ldots, x_{nd}]^T$ denotes the n-th continuous real-valued image vector. In an unsupervised auto-association (or autoencoding) paradigm, the target output is identical to the input:

$$\mathbf{y}_n = \mathbf{x}_n, \quad \forall n \in \{1, 2, \ldots, N\} \tag{2.2}$$

The central objective is to learn an encoder mapping that compresses the input $\mathbf{x}_n \in \mathbb{R}^d$ into a lower-dimensional latent representation $\mathbf{h}_n \in \mathbb{R}^l$ (where $l \ll d$), and a decoder mapping that reconstructs the original vector $\hat{\mathbf{x}}_n \in \mathbb{R}^d$ from $\mathbf{h}_n$ such that the reconstruction error is minimized.

2.2 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is an orthogonal linear transformation that converts a set of observations of possibly correlated variables into a set of values of linearly uncorrelated variables called principal components.

2.2.1 Mean-Centering and Covariance Computation

Given the training observations $\{\mathbf{x}_n\}_{n=1}^N$, the sample mean vector $\boldsymbol{\mu} \in \mathbb{R}^d$ is defined as:

$$\boldsymbol{\mu} = \frac{1}{N}\sum_{n=1}^{N}\mathbf{x}_n = \left[\frac{1}{N}\sum_{n=1}^{N}x_{n1}, \frac{1}{N}\sum_{n=1}^{N}x_{n2}, \ldots, \frac{1}{N}\sum_{n=1}^{N}x_{nd}\right]^T \tag{2.3}$$

Each sample $\mathbf{x}_n$ is mean-subtracted to obtain zero-mean representations:

$$\mathbf{z}_n = \mathbf{x}_n - \boldsymbol{\mu} = [x_{n1} - \mu_1, x_{n2} - \mu_2, \ldots, x_{nd} - \mu_d]^T \in \mathbb{R}^d \tag{2.4}$$

Collecting these vectors as rows into the mean-centered data matrix $\mathbf{Z} \in \mathbb{R}^{N\times d}$:

$$\mathbf{Z} = \begin{bmatrix}\mathbf{z}_1^T \\ \mathbf{z}_2^T \\ \vdots \\ \mathbf{z}_N^T\end{bmatrix} = \begin{bmatrix} z_{11} & z_{12} & \cdots & z_{1d} \\ z_{21} & z_{22} & \cdots & z_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ z_{N1} & z_{N2} & \cdots & z_{Nd}\end{bmatrix} \tag{2.5}$$

The sample covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{d\times d}$ is computed as:

$$\boldsymbol{\Sigma} = \frac{1}{N}\mathbf{Z}^T\mathbf{Z} = \frac{1}{N}\sum_{n=1}^{N}\mathbf{z}_n\mathbf{z}_n^T \tag{2.6}$$

where each element $\Sigma_{jk} = \frac{1}{N}\sum_{n=1}^{N} z_{nj}z_{nk}$ represents the empirical covariance between features j and k.

2.2.2 Eigendecomposition and Direction of Projection

Because $\boldsymbol{\Sigma}$ is real and symmetric, spectral decomposition yields d orthonormal eigenvectors $\mathbf{q}_i \in \mathbb{R}^d$ and real, non-negative eigenvalues $\lambda_i \geq 0$:

$$\boldsymbol{\Sigma}\mathbf{q}_i = \lambda_i\mathbf{q}_i, \quad i \in \{1, 2, \ldots, d\} \tag{2.7}$$

subject to the orthonormality constraint:

$$\mathbf{q}_i^T\mathbf{q}_j = \delta_{ij} = \begin{cases}1, & \text{if } i = j \\ 0, & \text{if } i \neq j\end{cases} \tag{2.8}$$

The eigenvalues are sorted in descending order of magnitude:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_l \geq \cdots \geq \lambda_d \geq 0 \tag{2.9}$$

The eigenvalue λ_i represents the variance of the data projected onto the i-th principal axis $\mathbf{q}_i$.

2.2.3 Dimension Reduction and Projection

To reduce dimensionality to l ($l < d$), we retain the top l leading eigenvectors to form the projection matrix:

$$\mathbf{Q}_l = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_l] \in \mathbb{R}^{d \times l} \tag{2.10}$$

The i-th principal component score a_{ni} for exemplar $\mathbf{x}_n$ is obtained via the inner product:

$$a_{ni} = \mathbf{q}_i^T \mathbf{z}_n = \mathbf{q}_i^T (\mathbf{x}_n - \boldsymbol{\mu}) = \sum_{j=1}^{d} q_{ji}(x_{nj} - \mu_j) \tag{2.11}$$

The full reduced-dimension representation vector $\mathbf{a}_n \in \mathbb{R}^l$ is:

$$\mathbf{a}_n = [a_{n1}, a_{n2}, \ldots, a_{nl}]^T = \mathbf{Q}_l^T \mathbf{z}_n = \mathbf{Q}_l^T (\mathbf{x}_n - \boldsymbol{\mu}) \tag{2.12}$$

Critical Implementation Note: In accordance with rigorous machine learning protocols, the projection directions $\mathbf{Q}_l$ and the mean vector $\boldsymbol{\mu}$ are computed exclusively from the *training* split. The validation and test samples are mean-centered using this training mean $\boldsymbol{\mu}_{\text{train}}$ before projection:

$$\mathbf{a}_n^{(\text{val})} = \mathbf{Q}_l^T (\mathbf{x}_n^{(\text{val})} - \boldsymbol{\mu}_{\text{train}}), \quad \mathbf{a}_n^{(\text{test})} = \mathbf{Q}_l^T (\mathbf{x}_n^{(\text{test})} - \boldsymbol{\mu}_{\text{train}}) \tag{2.13}$$

2.2.4 Reconstruction and Cumulative Retained Variance

Given the compressed representation $\mathbf{a}_n$, the reconstructed zero-mean signal $\hat{\mathbf{z}}_n$ and original image $\hat{\mathbf{x}}_n$ are recovered as:

$$\hat{\mathbf{z}}_n = \sum_{i=1}^{l} a_{ni}\mathbf{q}_i = \mathbf{Q}_l \mathbf{a}_n, \quad \hat{\mathbf{x}}_n = \hat{\mathbf{z}}_n + \boldsymbol{\mu} = \mathbf{Q}_l \mathbf{Q}_l^T (\mathbf{x}_n - \boldsymbol{\mu}) + \boldsymbol{\mu} \tag{2.14}$$

If all $d = 784$ components are preserved ($l = d$), the reconstruction is loss-free ($\hat{\mathbf{x}}_n = \mathbf{x}_n$). For $l < d$, the reconstruction is lossy, and the fraction of total variance retained by the l-dimensional subspace is given by:

$$\eta_l = \frac{\sum_{i=1}^{l} \lambda_i}{\sum_{j=1}^{d} \lambda_j} \times 100\% \tag{2.15}$$

2.3 Autoencoders (Auto-Associative Neural Networks)

An autoencoder is a specialized feed-forward neural network trained in an unsupervised manner to reproduce its input at its output layer through an informational bottleneck.

2.3.1 Architectural Components: Encoder and Decoder

An autoencoder comprises two distinct functional transformations:

1. **Encoder Function $g(\cdot)$:** Maps the high-dimensional input $\mathbf{x}_n \in \mathbb{R}^d$ into a latent code $\mathbf{h}_n \in \mathbb{R}^l$:

$$\mathbf{h}_n = g\left(\mathbf{W}^T\mathbf{x}_n + \mathbf{w}_0^{(e)}\right) \tag{2.16}$$

   where $\mathbf{W} \in \mathbb{R}^{d\times l}$ is the encoder weight matrix, $\mathbf{w}_0^{(e)} \in \mathbb{R}^l$ is the encoder bias vector, and $g(\cdot)$ is the activation function of the bottleneck neurons. Per the assignment specifications, the bottleneck layer is strictly a **linear layer**:

$$g(u) = u \implies \mathbf{h}_n = \mathbf{W}^T\mathbf{x}_n + \mathbf{w}_0^{(e)} \tag{2.17}$$

2. **Decoder Function $f(\cdot)$:** Maps the latent code $\mathbf{h}_n \in \mathbb{R}^l$ back into a reconstructed vector $\hat{\mathbf{x}}_n \in \mathbb{R}^d$:

$$\hat{\mathbf{x}}_n = f\left(\hat{\mathbf{W}}^T\mathbf{h}_n + \mathbf{w}_0^{(d)}\right) \tag{2.18}$$

   where $\hat{\mathbf{W}} \in \mathbb{R}^{l\times d}$ is the decoder weight matrix, $\mathbf{w}_0^{(d)} \in \mathbb{R}^d$ is the decoder bias vector, and $f(\cdot)$ is the output activation function. Because input pixels are normalized to $[0, 1]$, the output layer utilizes the standard logistic sigmoid activation:

$$f(u) = \sigma(u) = \frac{1}{1+e^{-u}} \tag{2.19}$$

2.3.2 Symmetric Multi-Hidden-Layer Autoencoders

Autoencoders can be constructed with an arbitrary odd number of hidden layers M. In an M-layer autoencoder (excluding input and output layers):

- The architecture is symmetric about the middle bottleneck layer: the number of neurons in layer m is identical to that in layer $(M - m + 1)$.
- The middle layer $((M + 1)/2)$ functions as the dimension reduction bottleneck.

In this investigation, we study two symmetric configurations:

1. **1-Hidden-Layer Autoencoder (AE-1):**

$$784 \xrightarrow{\text{Linear}} l \xrightarrow{\text{Sigmoid}} 784 \tag{2.20}$$

   The input is directly projected into the bottleneck $\mathbf{h}_n \in \mathbb{R}^l$ and subsequently decoded to $\hat{\mathbf{x}}_n \in \mathbb{R}^{784}$.

2. **3-Hidden-Layer Autoencoder (AE-3):**

$$784 \xrightarrow[\text{Sigmoid}]{\mathbf{W}_1} 400 \xrightarrow[\text{Linear}]{\mathbf{W}_2} l \xrightarrow[\text{Sigmoid}]{\mathbf{W}_3} 400 \xrightarrow[\text{Sigmoid}]{\mathbf{W}_4} 784 \tag{2.21}$$

The first hidden layer utilizes 400 sigmoidal neurons:

$$\mathbf{h}_n^{(1)} = \sigma\left(\mathbf{W}_1^T \mathbf{x}_n + \mathbf{b}_1\right), \quad \mathbf{h}_n^{(1)} \in \mathbb{R}^{400} \tag{2.22}$$

The bottleneck layer linearly compresses this intermediate representation:

$$\mathbf{h}_n = \mathbf{W}_2^T \mathbf{h}_n^{(1)} + \mathbf{b}_2, \quad \mathbf{h}_n \in \mathbb{R}^l \tag{2.23}$$

The third layer decodes the representation through 400 sigmoidal neurons:

$$\mathbf{h}_n^{(3)} = \sigma\left(\mathbf{W}_3^T \mathbf{h}_n + \mathbf{b}_3\right), \quad \mathbf{h}_n^{(3)} \in \mathbb{R}^{400} \tag{2.24}$$

Finally, the output layer generates the pixel reconstruction:

$$\hat{\mathbf{x}}_n = \sigma\left(\mathbf{W}_4^T \mathbf{h}_n^{(3)} + \mathbf{b}_4\right), \quad \hat{\mathbf{x}}_n \in \mathbb{R}^{784} \tag{2.25}$$

2.3.3 Objective Function and Reconstruction Error

Because the input features represent continuous grayscale intensities, the model is trained by minimizing the empirical Mean Squared Error (MSE) loss function:

$$\mathcal{L}_{\text{MSE}}(\boldsymbol{\Theta}) = \frac{1}{N} \sum_{n=1}^{N} \sum_{i=1}^{d} (x_{ni} - \hat{x}_{ni})^2 = \frac{1}{N} \sum_{n=1}^{N} \|\mathbf{x}_n - \hat{\mathbf{x}}_n\|^2 \tag{2.26}$$

where $\boldsymbol{\Theta} = \{\mathbf{W}, \hat{\mathbf{W}}, \mathbf{w}_0^{(e)}, \mathbf{w}_0^{(d)}\}$ denotes the complete set of trainable parameters. After training converges, the average reconstruction error per sample on any dataset split $\mathcal{S}$ is reported as:

$$\text{MSE}(\mathcal{S}) = \frac{1}{|\mathcal{S}| \cdot d} \sum_{n \in \mathcal{S}} \sum_{i=1}^{d} (x_{ni} - \hat{x}_{ni})^2 \tag{2.27}$$

2.4 Theoretical Link between PCA and Autoencoders

A profound theoretical connection exists between Principal Component Analysis and linear autoencoders, established in classical neural network literature by Bourlard and Kamp (1988) as well as Baldi and Hornik (1989).

[Equivalence of Linear Autoencoders and PCA Subspaces] Let an autoencoder possess a single hidden layer of l linear units ($g(u) = u$) and a linear output layer ($f(u) = u$). Let the network be trained to minimize the mean squared error objective:

$$\mathcal{L}(\mathbf{W}, \hat{\mathbf{W}}) = \frac{1}{N} \sum_{n=1}^{N} \|\mathbf{x}_n - \hat{\mathbf{W}}\mathbf{W}^T \mathbf{x}_n\|^2 \tag{2.28}$$

over a zero-mean dataset $\mathbf{Z}$ ($\boldsymbol{\mu} = \mathbf{0}$). Then:

1. The global minimum of the loss function is attained if and only if the l-dimensional subspace spanned by the column vectors of the encoder weight matrix $\mathbf{W} \in \mathbb{R}^{d \times l}$ is identical to the principal subspace spanned by the first l eigenvectors $\{\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_l\}$ of the sample covariance matrix $\mathbf{\Sigma}$.

2. The minimum reconstruction error achieved by the linear autoencoder matches the reconstruction error of PCA:

$$\min_{\mathbf{W}, \hat{\mathbf{W}}} \mathcal{L}(\mathbf{W}, \hat{\mathbf{W}}) = \sum_{j=l+1}^{d} \lambda_j \tag{2.29}$$

While individual weight vectors in $\mathbf{W}$ do not necessarily correspond to individual eigenvectors $\mathbf{q}_i$ (due to arbitrary orthogonal rotational freedom $\mathbf{WR}$ where $\mathbf{R} \in O(l)$), their linear span is identical.

Advantages of Nonlinear Autoencoders: When sigmoidal or tanh activation functions are introduced into the intermediate layers of deep autoencoders (as in AE-3), the network is no longer restricted to linear subspaces. Nonlinear autoencoders approximate nonlinear coordinate systems, learning to unfold curved manifolds that cannot be captured by planar PCA projections without severe distortion.

2.5 Denoising Autoencoders (DAE)

While standard undercomplete autoencoders rely solely on dimension reduction to prevent trivial identity mappings, an alternative, powerful regularization paradigm is the Denoising Autoencoder (DAE), introduced by Vincent et al. (2008).

2.5.1 Stochastic Corruption Process

In a denoising autoencoder, the clean training sample $\mathbf{x}_n$ is partially corrupted through a stochastic process $q_D(\tilde{\mathbf{x}}_n|\mathbf{x}_n)$ before being presented to the encoder. For each pixel $i \in \{1, 2, \ldots, d\}$, independent noise is applied with probability $q \in (0, 1)$:

$$\tilde{x}_{ni} = \begin{cases} \text{clip}\left(x_{ni} + \epsilon_{ni}, 0, 1\right), & \text{with probability } q \\ x_{ni}, & \text{with probability } 1 - q \end{cases} \tag{2.30}$$

where $\epsilon_{ni} \sim \mathcal{N}(0, \sigma^2)$ is zero-mean Gaussian perturbation with standard deviation $\sigma = 0.5$, and $\text{clip}(v, 0, 1) = \max(0, \min(1, v))$ ensures valid pixel boundaries. With probability $1-q$, the pixel is retained without alteration. Consequently, across training epochs, different subsets of pixels are corrupted for each exemplar, preventing the network from memorizing individual pixel values.

2.5.2 Denoising Objective and Manifold Learning

The corrupted representation $\tilde{\mathbf{x}}_n$ is mapped through the encoder to obtain latent code $\mathbf{h}_n$:

$$\mathbf{h}_n = g\left(\mathbf{W}^T\tilde{\mathbf{x}}_n + \mathbf{w}_0^{(e)}\right) \tag{2.31}$$

Crucially, the decoder is tasked with reconstructing the **uncorrupted, clean** original image $\mathbf{x}_n$:

$$\hat{\mathbf{x}}_n = f\left(\hat{\mathbf{W}}^T\mathbf{h}_n + \mathbf{w}_0^{(d)}\right) \tag{2.32}$$

The optimization objective is:

$$\mathcal{L}_{\text{DAE}}(\boldsymbol{\Theta}) = \frac{1}{N}\sum_{n=1}^{N}\|\mathbf{x}_n - \hat{\mathbf{x}}_n(\tilde{\mathbf{x}}_n)\|^2 \tag{2.33}$$

To minimize this loss, the network cannot rely on trivial identity mappings. It is forced to learn the statistical dependencies and spatial correlations among adjacent pixels, effectively learning an orthogonal vector field that projects corrupted off-manifold points back onto the true low-dimensional data manifold.

2.6 Weight Visualization and Receptive Fields

Understanding the internal representations learned by neural networks is critical for interpretability. In this work, we visualize the learned weights connecting the 784-dimensional input layer to the hidden neurons.

2.6.1 Mathematical Derivation of Maximal Activation Stimulus

Let $\mathbf{W} \in \mathbb{R}^{d\times l}$ be the encoder weight matrix connecting the $d = 784$ input pixels to the l bottleneck neurons:

$$\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_l], \quad \mathbf{w}_j = [w_{1j}, w_{2j}, \ldots, w_{dj}]^T \in \mathbb{R}^d \tag{2.34}$$

The pre-activation net input to the j-th hidden neuron for an input $\mathbf{x}$ is:

$$a_j = \mathbf{w}_j^T\mathbf{x} + w_{0j}^{(e)} = \sum_{i=1}^{d} w_{ij}x_i + w_{0j}^{(e)} \tag{2.35}$$

We seek the input vector $\mathbf{x}^*$ that maximally activates neuron j, subject to a finite-energy constraint $\|\mathbf{x}\|_2 \leq 1$. By the Cauchy-Schwarz inequality:

$$\mathbf{w}_j^T\mathbf{x} \leq \|\mathbf{w}_j\|_2\|\mathbf{x}\|_2 \leq \|\mathbf{w}_j\|_2 \tag{2.36}$$

with equality holding if and only if $\mathbf{x}$ is collinear with $\mathbf{w}_j$:

$$\mathbf{x}^* = \frac{\mathbf{w}_j}{\|\mathbf{w}_j\|_2} \tag{2.37}$$

Hence, the weight vector $\mathbf{w}_j$ itself directly defines the optimal template or receptive field filter that maximally excites neuron j. By reshaping $\mathbf{w}_j \in \mathbb{R}^{784}$ into a 2D matrix of shape 28×28, we can visualize the spatial pattern to which neuron j is tuned.

2.6.2 Quantitative Filter Sharpness via Laplacian Variance

To quantify the spatial focus and sharpness of the learned receptive fields, we compute the discrete 2D Laplacian of each filter matrix $\mathbf{W}_j \in \mathbb{R}^{28\times 28}$:

$$\Delta\mathbf{W}_j(u,v) = \mathbf{W}_j(u+1,v) + \mathbf{W}_j(u-1,v) + \mathbf{W}_j(u,v+1) + \mathbf{W}_j(u,v-1) - 4\mathbf{W}_j(u,v) \tag{2.38}$$

The filter sharpness metric is defined as the sample variance of the Laplacian response:

$$\text{Sharpness}(\mathbf{w}_j) = \text{Var}\left(\Delta\mathbf{W}_j\right) = \frac{1}{28\times 28}\sum_{u=1}^{28}\sum_{v=1}^{28}\left(\Delta\mathbf{W}_j(u,v) - \overline{\Delta\mathbf{W}}_j\right)^2 \tag{2.39}$$

High Laplacian variance indicates rapid spatial transitions corresponding to high-contrast edges, whereas low variance indicates smooth, diffuse, or global structures.

2.7 Downstream Classification using Fully Connected Neural Networks

To rigorously evaluate the discriminative utility of each learned representation, the compressed vectors ($\mathbf{a}_n$ from PCA or $\mathbf{h}_n$ from autoencoders) are used as inputs to a multi-class Fully Connected Neural Network (FCNN).

2.7.1 Network Architecture and Forward Propagation

Let the input representation dimension be $l \in \{32, 64, 128, 256\}$. The FCNN comprises L fully connected hidden layers followed by an output layer of $K = 5$ neurons corresponding to digit classes $\{3, 4, 5, 7, 8\}$:

$$\mathbf{z}^{(1)} = \tanh\left(\mathbf{V}_1^T\mathbf{h}_n + \mathbf{b}_1\right) \tag{2.40}$$

$$\mathbf{z}^{(k)} = \tanh\left(\mathbf{V}_k^T\mathbf{z}^{(k-1)} + \mathbf{b}_k\right), \quad k \in \{2, 3, \ldots, L\} \tag{2.41}$$

$$\mathbf{s}_n = \mathbf{V}_{\text{out}}^T\mathbf{z}^{(L)} + \mathbf{b}_{\text{out}} \in \mathbb{R}^K \tag{2.42}$$

where the hyperbolic tangent activation function is:

$$\tanh(u) = \frac{e^{u} - e^{-u}}{e^{u} + e^{-u}} \tag{2.43}$$

The predicted class probability distribution $\hat{\mathbf{p}}_n \in \mathbb{R}^K$ is produced via the Softmax operator:

$$\hat{p}_{nc} = P(y_n = c \mid \mathbf{h}_n) = \frac{\exp(s_{nc})}{\sum_{k=1}^{K} \exp(s_{nk})}, \quad c \in \{1, 2, \ldots, K\} \tag{2.44}$$

2.7.2 Cross-Entropy Objective and Gradient Descent Optimization

The classification network is trained by minimizing the categorical cross-entropy loss:

$$\mathcal{L}_{\text{CE}}(\mathbf{\Phi}) = -\frac{1}{N} \sum_{n=1}^{N} \sum_{c=1}^{K} y_{nc} \log \hat{p}_{nc} \tag{2.45}$$

where $y_{nc} \in \{0, 1\}$ is the one-hot ground-truth label for sample n. To ensure strict consistency with the best-performing configuration from Assignment 3, the classifiers are optimized using Stochastic Gradient Descent with Momentum (batch size = 1, learning rate $\eta = 0.001$, momentum coefficient $\gamma = 0.9$):

$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + \eta \nabla_{\mathbf{\Phi}} \mathcal{L}_{\text{CE}}(\mathbf{\Phi}_t) \tag{2.46}$$

$$\mathbf{\Phi}_{t+1} = \mathbf{\Phi}_t - \mathbf{v}_t \tag{2.47}$$

Weights are initialized deterministically using Xavier normal initialization with zero initial biases.

Chapter 3

Experimental Setup

This chapter details the dataset partition, implementation specifics, network architectures, parameter configurations, and evaluation metrics employed across all experimental tasks.

3.1 Dataset Description and Preprocessing

The empirical investigation is conducted on a five-class partition of the MNIST database of handwritten digits, specifically assigned to **Group 30**. The target classes comprise digits:

$$\mathcal{C} = \{3, 4, 5, 7, 8\} \tag{3.1}$$

The dataset is pre-partitioned into training, validation, and test subsets with the distribution summarized in Table 3.1.

Table 3.1: Distribution of samples per class across dataset splits for Group 30.

Dataset Split	**Digit 3**	**Digit 4**	**Digit 5**	**Digit 7**	**Digit 8**	**Total Samples**
Training Set	2,277	2,277	2,277	2,277	2,277	11,385 (60.0%)
Validation Set	759	759	759	759	759	3,795 (20.0%)
Test Set	759	759	759	759	759	3,795 (20.0%)
Total	3,795	3,795	3,795	3,795	3,795	**18,975 (100.0%)**

Each sample is originally an 8-bit grayscale image of spatial dimensions 28×28 pixels. Each image is unrolled (flattened) into a vector of length $d = 784$ with pixel values normalized to the real interval $[0.0, 1.0]$:

$$x_i = \frac{\text{pixel_intensity}_i}{255.0}, \quad \forall i \in \{1, 2, \ldots, 784\} \tag{3.2}$$

Figure 3.1 displays representative original sample images directly extracted from the training directory for each of the five assigned classes.

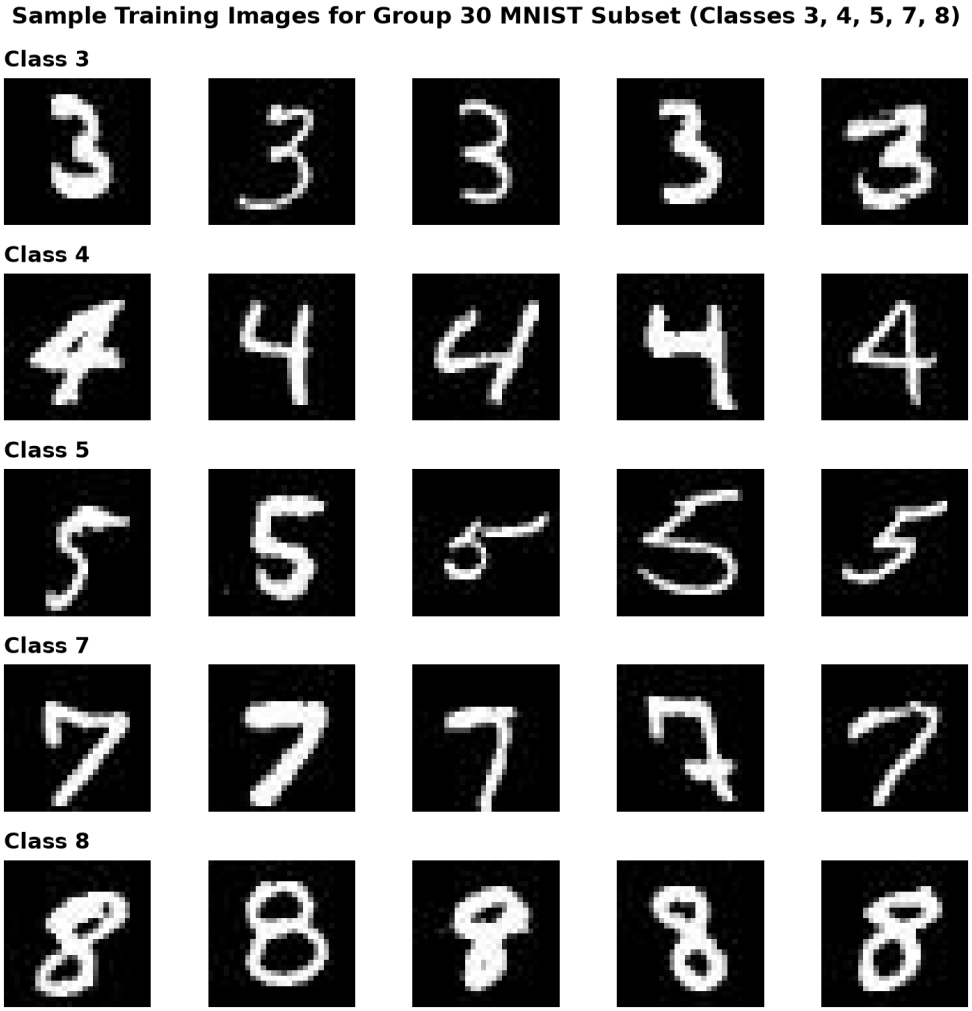


Figure 3.1: Representative sample handwritten digits from the Group 30 training dataset across the five classes ($\{3, 4, 5, 7, 8\}$).

3.2 PCA Configuration

For Principal Component Analysis (Task 1):

1. The mean vector $\boldsymbol{\mu}_{\text{train}} \in \mathbb{R}^{784}$ and the sample covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{784\times784}$ are estimated strictly from the 11,385 training exemplars.

2. Full eigendecomposition yields 784 orthonormal eigenvectors $\mathbf{q}_i$.

3. Reduced representations of dimension $l \in \{32, 64, 128, 256\}$ are produced by selecting the top l eigenvectors $\mathbf{Q}_l = [\mathbf{q}_1, \ldots, \mathbf{q}_l]$.

4. Mean-subtraction for validation and test samples uses $\boldsymbol{\mu}_{\text{train}}$ before linear projection $\mathbf{a}_n = \mathbf{Q}_l^T(\mathbf{x}_n - \boldsymbol{\mu}_{\text{train}})$.

3.3 Autoencoder Architectures and Training Hyperparameters

In Task 2, two autoencoder topologies are implemented:

- **1-Hidden-Layer Autoencoder (AE-1):** Structured as $784 \to l \to 784$. The bottleneck layer consists of l linear units, and the output layer consists of 784 sigmoidal units.
- **3-Hidden-Layer Autoencoder (AE-3):** Structured as $784 \to 400 \to l \to 400 \to 784$. The first and third hidden layers contain 400 logistic sigmoidal units, the bottleneck contains l linear units, and the output layer contains 784 sigmoidal units.

Four bottleneck dimensions are evaluated: $l \in \{32, 64, 128, 256\}$. Table 3.2 provides the exact parameter counts for all eight autoencoder models.

Table 3.2: Trainable parameter counts for 1-hidden-layer (AE-1) and 3-hidden-layer (AE-3) autoencoders across bottleneck dimensions.

Autoencoder Model	$l = 32$	$l = 64$	$l = 128$	$l = 256$
AE-1 ($784 \to l \to 784$)	50,992	101,200	201,616	402,448
AE-3 ($784 \to 400 \to l \to 400 \to 784$)	654,416	680,048	731,312	833,840

Autoencoder Training Protocol: All autoencoders are trained using the Adam optimizer with the following hyperparameter specification:

- Mini-batch size: $B = 128$.
- Initial learning rate: $\eta = 0.001$.
- First moment decay rate: $\beta_1 = 0.9$.
- Second moment decay rate: $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.
- Loss criterion: Mean Squared Error (MSE).

- Convergence and Early Stopping: Training is monitored on validation MSE. If validation MSE fails to improve by at least $\Delta_{\min} = 10^{-5}$ over a patience window of 5 epochs, training terminates, and the model weights yielding the lowest validation MSE are restored.

3.4 Denoising Autoencoder Configuration

In Task 5, 1-hidden-layer Denoising Autoencoders are constructed at the optimal bottleneck dimension identified in Task 3 ($l = 32$).

- Noise model: Independent additive Gaussian perturbation $\epsilon \sim \mathcal{N}(0, 0.5^2)$ clipped to $[0.0, 1.0]$.
- Noise probability levels: $q = 0.20$ (20% noise) and $q = 0.40$ (40% noise).
- Training objective: Reconstruct the clean, uncorrupted input $\mathbf{x}_n$ from the noisy input $\tilde{\mathbf{x}}_n$.
- Training hyperparameters: Adam optimizer, $\eta = 0.001$, batch size 128, patience 5 epochs.

3.5 FCNN Classifier Architectures and Training Setup

In Tasks 1, 3, 4, and 5, multi-class pattern classification is conducted on the reduced representations using nine candidate Fully Connected Neural Network (FCNN) architectures, spanning 3-layer, 4-layer, and 5-layer topologies. Table 3.3 specifies the hidden layer dimensions and total parameter counts for each input dimensionality ($l \in \{32, 64, 128, 256\}$ as well as the raw 784-dimensional pixel baseline).

Classifier Optimizer and Hyperparameters: To establish direct comparability with Assignment 3, all FCNN classifiers are trained under the identical optimal optimizer configuration determined in Assignment 3:

- **Algorithm:** Stochastic Gradient Descent with Momentum (SGD + Momentum).
- **Batch Size:** $B = 1$ (pure online stochastic gradient updates).
- **Learning Rate:** $\eta = 0.001$.
- **Momentum Coefficient:** $\gamma = 0.9$.
- **Hidden Activation:** Hyperbolic tangent (tanh).

Table 3.3: FCNN classifier architecture specifications and total parameter counts across input dimensions.

Architecture	**Hidden Layer Units**	$l = 32$	$l = 64$	$l = 128$	$l = 256$	$d = 784$
3-Hidden-Layer Models						
`arch_3l_v1`	$[512, 256, 128]$	181,765	198,149	230,917	296,453	566,789
`arch_3l_v2`	$[256, 128, 64]$	49,925	58,117	74,501	107,269	242,437
`arch_3l_v3`	$[256, 256, 256]$	141,317	149,509	165,893	198,661	333,829
4-Hidden-Layer Models						
`arch_4l_v1`	$[512, 256, 128, 64]$	189,701	206,085	238,853	304,389	574,725
`arch_4l_v2`	$[256, 128, 64, 32]$	51,845	60,037	76,421	109,189	244,357
`arch_4l_v3`	$[384, 256, 128, 64]$	152,709	164,997	189,573	238,725	441,477
5-Hidden-Layer Models						
`arch_5l_v1`	$[512, 384, 256, 128, 64]$	353,925	370,309	403,077	468,613	738,949
`arch_5l_v2`	$[256, 256, 128, 128, 64]$	132,229	140,421	156,805	189,573	324,741
`arch_5l_v3`	$[256, 128, 64, 64, 32]$	56,005	64,197	80,581	113,349	248,517

- **Output Activation and Loss:** Softmax with multi-class Cross-Entropy loss.
- **Weight Initialization:** Xavier normal initialization with seed = 42; biases initialized to zero.
- **Stopping Threshold:** Absolute loss change threshold of 10^{-4} with patience = 1 epoch, up to a maximum of 10,000 epochs.

Assignment 3 Baseline Reference: In Assignment 3, the best-performing model trained directly on raw 784-dimensional pixels was architecture `arch_4l_v3` ($[384, 256, 128, 64]$) using SGD + Momentum ($\eta = 0.001, \gamma = 0.9, B = 1$). Its benchmark metrics were:

$$\text{Val Accuracy} = 98.39\%, \quad \text{Test Accuracy} = 98.02\%, \quad \text{Test Macro-F1} = 98.02\% \tag{3.3}$$

All downstream classification results in Tasks 1, 3, 4, and 5 are compared directly against this baseline.

3.6 Evaluation Metrics

Model performance is evaluated using the following formal metrics:

1. **Classification Accuracy (%):** The percentage of correctly predicted instances over the total number of evaluated samples:

$$\text{Accuracy} = \frac{\sum_{c=1}^{K} TP_c}{N_{\text{eval}}} \times 100\% \tag{3.4}$$

where TP_c is the number of true positive classifications for class c, and $N_{\text{eval}} = 3,795$ for the test set.

2. **Macro-Averaged Precision, Recall, and F1-Score (%):**

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \tag{3.5}$$

$$\text{F1}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{3.6}$$

$$\text{Macro-F1} = \frac{1}{K} \sum_{c=1}^{K} \text{F1}_c \times 100\% \tag{3.7}$$

3. **Confusion Matrix:** A $K \times K$ contingency table $\mathbf{C}$ where entry C_{ij} denotes the number of samples with ground-truth class i predicted as class j.

4. **Reconstruction Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{N_{\text{eval}} \cdot 784} \sum_{n=1}^{N_{\text{eval}}} \|\mathbf{x}_n - \hat{\mathbf{x}}_n\|_2^2 \tag{3.8}$$

Chapter 4

Task 1: Dimension Reduction using PCA

This chapter presents the experimental results for Task 1: linear dimensionality reduction using Principal Component Analysis (PCA) and subsequent multi-class classification using Fully Connected Neural Networks (FCNNs).

4.1 Variance Retained by Principal Components

Eigendecomposition of the sample covariance matrix $\mathbf{\Sigma}_{\text{train}}$, computed exclusively from the 11,385 training samples, yields 784 eigenvalues. The cumulative retained variance ratio $\eta_l = \frac{\sum_{i=1}^{l} \lambda_i}{\sum_{j=1}^{784} \lambda_j} \times 100\%$ is reported in Table 4.1 for the four target dimensions.

Table 4.1: Cumulative variance retained by PCA on the Group 30 training dataset.

Reduced Dimension (l)	**Compression Factor**	**Cumulative Variance Retained (%)**
32	24.5×	73.81%
64	12.3×	85.77%
128	6.1×	93.46%
256	3.1×	97.96%

Retaining only 32 dimensions captures nearly three-quarters (73.81%) of the total image energy, while compressing the feature space by a factor of 24.5. Retaining 256 dimensions preserves almost 98% of the total variance.

4.2 Validation Accuracy across FCNN Architectures

For each reduced dimension $l \in \{32, 64, 128, 256\}$, all nine candidate FCNN architectures (defined in Chapter 3) are trained on the projected training samples and evaluated on the projected validation samples. Table 4.2 displays the complete validation accuracy grid.

Table 4.2: Validation classification accuracy (%) across nine FCNN architectures for each PCA dimension. Bold entries indicate the selected optimal architecture for each dimension.

Architecture	Hidden Layers	PCA-32	PCA-64	PCA-128	PCA-256
`arch_3l_v1`	[512, 256, 128]	98.21	98.29	98.29	97.97
`arch_3l_v2`	[256, 128, 64]	97.97	98.23	98.21	97.89
`arch_3l_v3`	[256, 256, 256]	**98.39**	**98.47**	98.29	98.23
`arch_4l_v1`	[512, 256, 128, 64]	98.37	98.10	98.10	98.16
`arch_4l_v2`	[256, 128, 64, 32]	97.84	98.42	**98.42**	98.02
`arch_4l_v3`	[384, 256, 128, 64]	98.23	98.21	98.26	98.10
`arch_5l_v1`	[512, 384, 256, 128, 64]	98.18	98.47	98.26	98.21
`arch_5l_v2`	[256, 256, 128, 128, 64]	98.16	98.13	98.34	**98.26**
`arch_5l_v3`	[256, 128, 64, 64, 32]	98.34	98.02	98.10	97.94

A visual heatmap of these validation accuracies is presented in Figure 4.1. Across all architectures, the validation performance remains remarkably stable above 97.8%, with `arch_3l_v3` ([256, 256, 256]) achieving peak validation accuracy for both 32 dimensions (98.39%) and 64 dimensions (98.47%).

4.3 Test Performance of the Selected Architectures

Based strictly on validation accuracy, the best architecture for each PCA dimension is selected and evaluated on the unseen test set of 3,795 samples. Table 4.3 summarizes the complete test performance metrics.

4.3.1 Convergence History and Confusion Matrices

The training loss histories and corresponding test confusion matrices for all four selected models are shown in Figures 4.2 through 4.5.

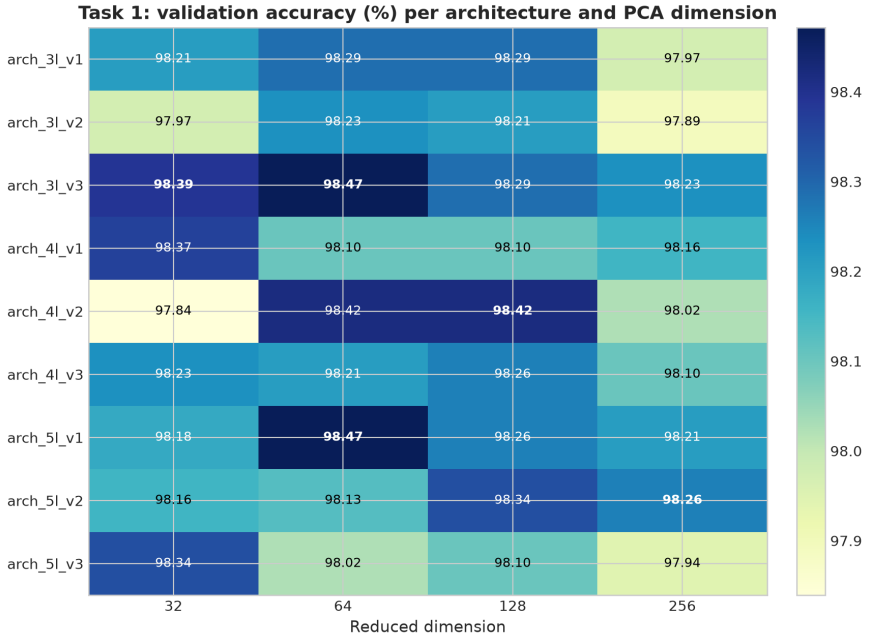


Figure 4.1: Validation accuracy heatmap for the nine FCNN architectures across the four PCA reduced dimensions.

Table 4.3: Performance on the test dataset for the best FCNN architectures selected per PCA dimension.

Dimension	Best Architecture	Val Acc (%)	Test Acc (%)	Macro-Precision (%)	Macro-F1 (%)
32	`arch_3l_v3` $[256, 256, 256]$	98.39	**98.47**	98.47	**98.47**
64	`arch_3l_v3` $[256, 256, 256]$	98.47	98.18	98.18	98.18
128	`arch_4l_v2` $[256, 128, 64, 32]$	98.42	98.00	98.00	98.00
256	`arch_5l_v2` $[256, 256, 128, 128, 64]$	98.26	97.81	97.81	97.81

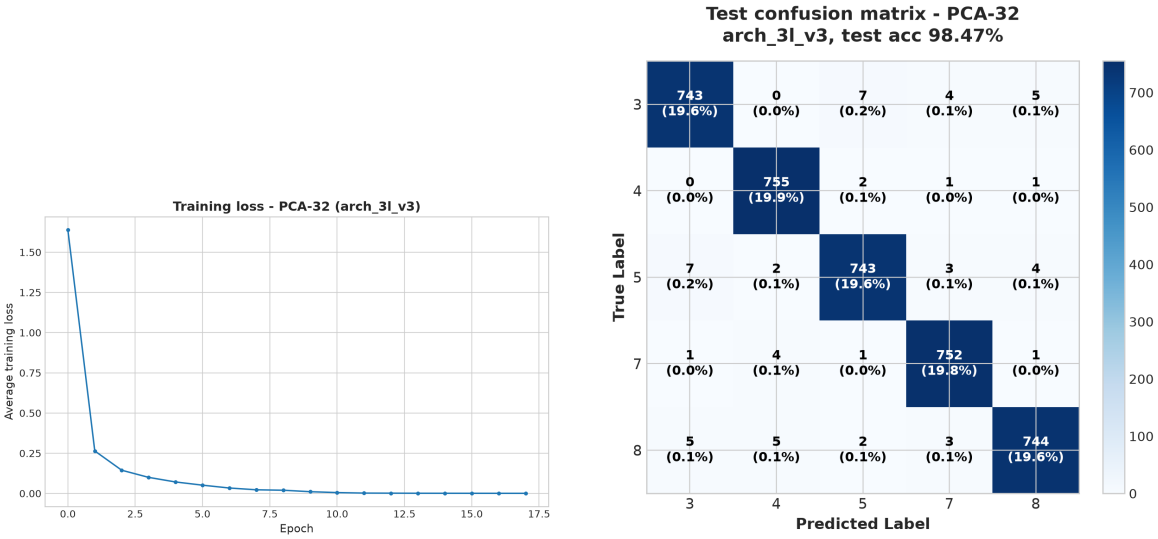
4.3.2 Detailed Error Analysis on Test Predictions

Examining the exact confusion matrix for the best model (PCA-32, `arch_3l_v3`):

$$\mathbf{C}_{\text{PCA-32}} = \begin{bmatrix} 743 & 0 & 7 & 4 & 5 \\ 0 & 755 & 2 & 1 & 1 \\ 7 & 2 & 743 & 3 & 4 \\ 1 & 4 & 1 & 752 & 1 \\ 5 & 5 & 2 & 3 & 744 \end{bmatrix} \tag{4.1}$$

where rows represent true classes and columns represent predictions for classes $\{3, 4, 5, 7, 8\}$.

- Digits **4** and **7** achieve the highest recognition rates (755/759 and 752/759 correctly classified, corresponding to 99.47% and 99.08% class recall, respectively) due to their distinctive sharp vertical and horizontal stroke angles.
- The predominant sources of error occur symmetrically between digits **3** and **5** (7



(a) Training loss convergence.

(b) Test confusion matrix.

Figure 4.2: Training loss trajectory and test confusion matrix for PCA-32 using `arch_3l_v3` ([256, 256, 256]).

samples of digit 3 misclassified as 5, and 7 samples of digit 5 misclassified as 3). Morphologically, both digits share an identical lower half-loop; when handwriting slants or compresses the upper horizontal bar of digit 5, linear PCA projection collapses these features into overlapping subspaces.

- Digit **8** exhibits occasional confusion with digits 3 (5 samples) and 4 (5 samples), attributable to closed versus open loop ambiguities.

4.4 Observation on the Best Reduced Dimension

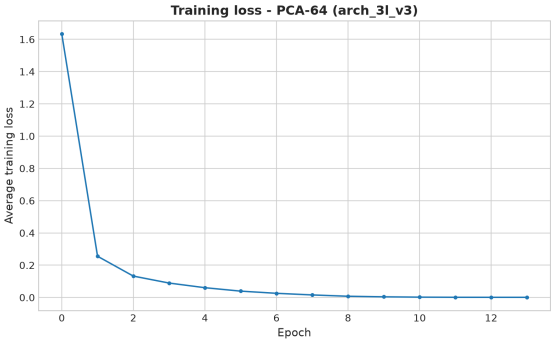
From Table 4.3 and Figure 4.6, we observe a clear monotonic inverse relationship: **as the number of retained principal components increases from 32 to 256, downstream test accuracy declines continuously**:

$$98.47\%\ (32\text{d}) \ > \ 98.18\%\ (64\text{d}) \ > \ 98.00\%\ (128\text{d}) \ > \ 97.81\%\ (256\text{d}) \tag{4.2}$$

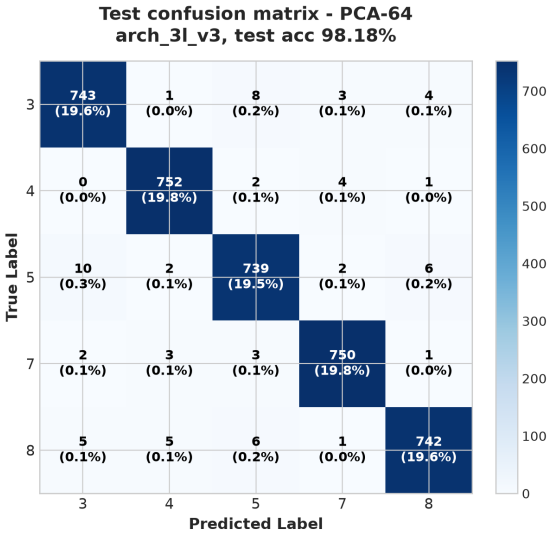
The optimal reduced dimension is unambiguously **32 dimensions**, attaining 98.47% test accuracy and 98.47% Macro-F1.

4.5 Comparison with Assignment 3 Best Result

Table 4.4 presents a direct comparison between the PCA reduced representations and the best baseline from Assignment 3.



(a) Training loss convergence.



(b) Test confusion matrix.

Figure 4.3: Training loss trajectory and test confusion matrix for PCA-64 using `arch_3l_v3` ($[256, 256, 256]$).

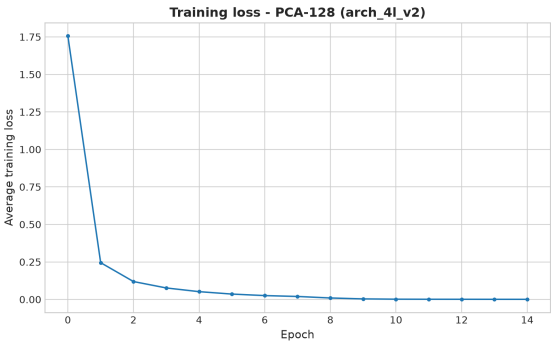
Table 4.4: Comparison of PCA classification performance against the best result of Assignment 3.

Model / Representation	Input Dim	Best Architecture	Val Acc (%)	Test Acc (%)	Δ vs. Baseline
Assignment 3 (Raw Pixels)	784	`arch_4l_v3` $[384, 256, 128, 64]$	98.39	98.02	Baseline
PCA-32	32	`arch_3l_v3` $[256, 256, 256]$	98.39	**98.47**	**+0.45%**
PCA-64	64	`arch_3l_v3` $[256, 256, 256]$	98.47	98.18	+0.16%
PCA-128	128	`arch_4l_v2` $[256, 128, 64, 32]$	98.42	98.00	-0.02%
PCA-256	256	`arch_5l_v2` $[256, 256, 128, 128, 64]$	98.26	97.81	-0.21%

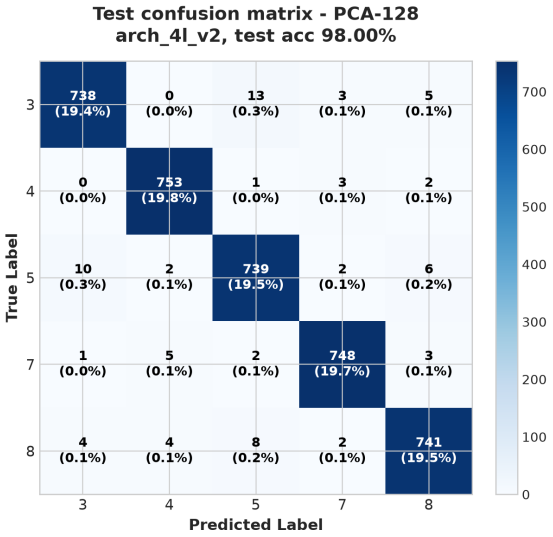
Analytical Insights:

1. **Denoising and Regularization Effect:** Retaining only 32 dimensions provides a substantial +0.45% boost in test accuracy over the uncompressed 784-dimensional baseline, despite discarding 26.19% of the variance. The trailing 752 dimensions primarily encode high-frequency background noise, border artifacts, and idiosyncratic stroke jitter. By discarding these trailing components, PCA acts as an effective linear low-pass spatial filter, preventing the downstream FCNN from overfitting to pixel noise.

2. **Curse of Overparameterization:** As the dimension expands to 128 and 256, the first layer of the FCNN requires significantly more parameters (e.g., expanding from 8,192 input weights at $l = 32$ to 65,536 weights at $l = 256$), which re-introduces noise sensitivity and causes test accuracy to drop below the Assignment 3 baseline (down to 97.81% at $l = 256$).

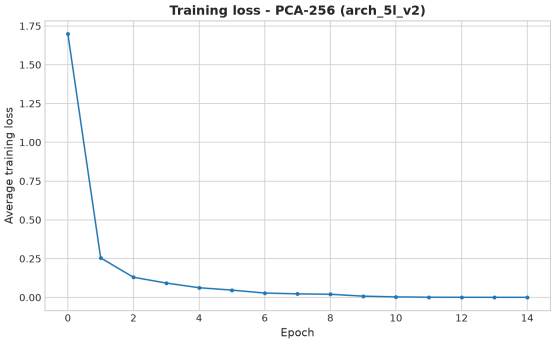


(a) Training loss convergence.

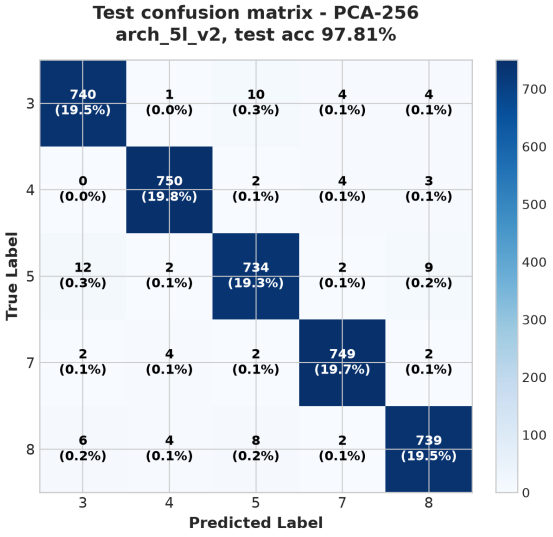


(b) Test confusion matrix.

Figure 4.4: Training loss trajectory and test confusion matrix for PCA-128 using `arch_4l_v2` ($[256, 128, 64, 32]$).



(a) Training loss convergence.



(b) Test confusion matrix.

Figure 4.5: Training loss trajectory and test confusion matrix for PCA-256 using `arch_5l_v2` ($[256, 256, 128, 128, 64]$).

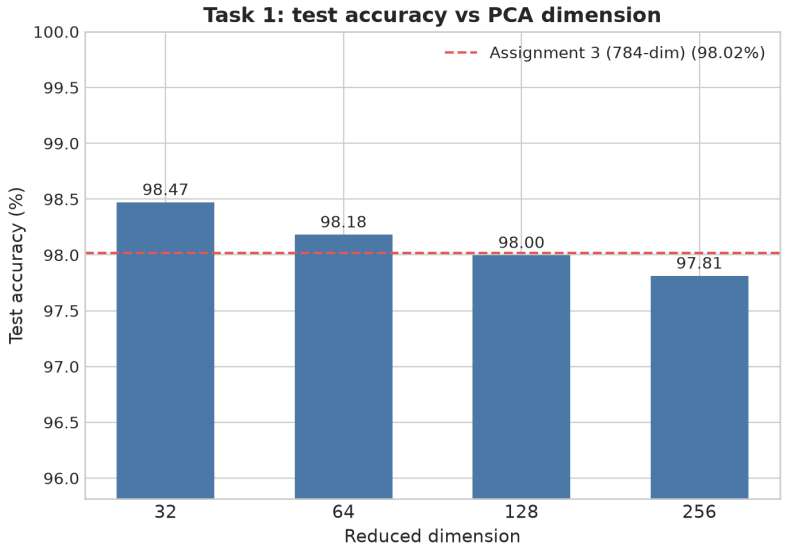


Figure 4.6: Test accuracy versus PCA reduced dimension, compared against the Assignment 3 raw pixel baseline (98.02%).

Chapter 5

Task 2: Autoencoders for Reconstructing Images

This chapter investigates unsupervised image reconstruction using shallow and deep autoencoder architectures across varying bottleneck dimensions, analyzing average reconstruction errors and qualitative visual fidelities.

5.1 One-Hidden-Layer Autoencoders (AE-1)

The shallow autoencoder follows a symmetric $784 \to l \to 784$ topology with a linear bottleneck layer and a sigmoidal reconstruction layer. Table 5.1 reports the training epochs to convergence and the resulting average Mean Squared Error (MSE) per pixel computed across the training, validation, and test datasets.

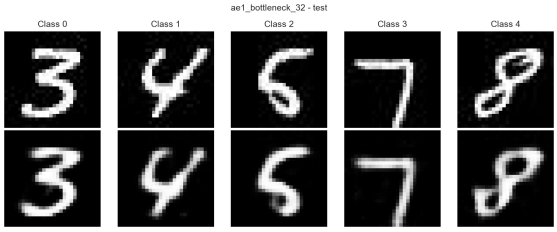
Table 5.1: Reconstruction performance (MSE) and training convergence for the 1-hidden-layer autoencoders (AE-1).

Bottleneck (l)	**Epochs**	**Train MSE**	**Validation MSE**	**Test MSE**
32	83	0.009258	0.010007	0.009970
64	91	0.003384	0.004000	0.003990
128	94	0.001314	0.001717	0.001715
256	97	**0.000693**	**0.000961**	**0.000963**

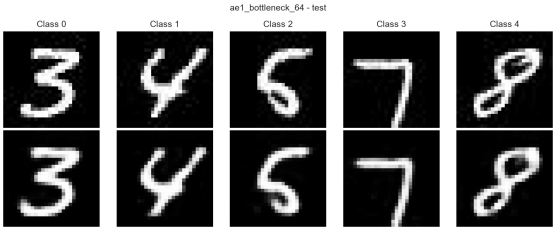
5.1.1 Visual Reconstruction Analysis for AE-1

Figures 5.1 presents the original versus reconstructed test images for one representative exemplar from each assigned class ($\{3, 4, 5, 7, 8\}$) across the four bottleneck dimensions.

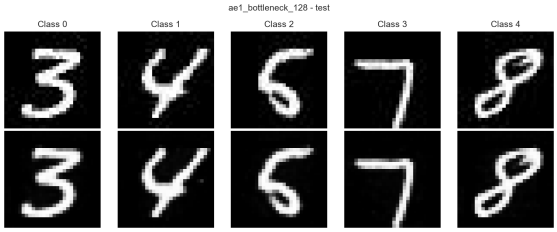
Additionally, Figure 5.2 shows the reconstructions on training and validation samples for the 32-dimensional bottleneck.



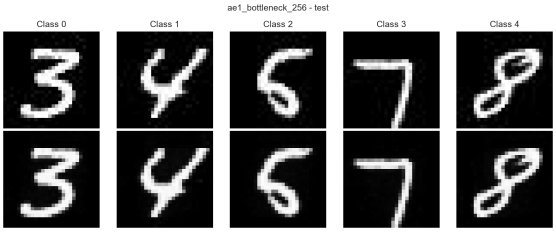
(a) $l = 32$ (Test MSE: 0.009970).



(b) $l = 64$ (Test MSE: 0.003990).

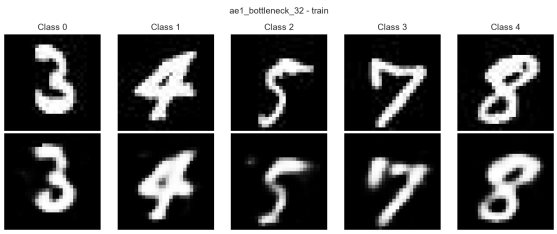


(c) $l = 128$ (Test MSE: 0.001715).

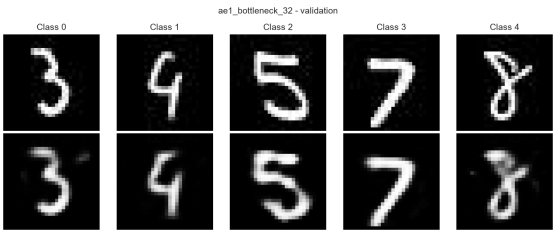


(d) $l = 256$ (Test MSE: 0.000963).

Figure 5.1: Reconstructions on test images for 1-hidden-layer autoencoders (AE-1) across bottleneck dimensions.



(a) AE-1 ($l = 32$) on Training Exemplars.



(b) AE-1 ($l = 32$) on Validation Exemplars.

Figure 5.2: Reconstruction fidelity on training and validation sets for 1-hidden-layer autoencoder with bottleneck $l = 32$.

At $l = 32$, the reconstructions retain the overall topological class structure but exhibit slight blurriness and faint halos around thin stroke junctions (e.g., the intersection in digit 4 and the central loop crossing in digit 8). As the bottleneck expands to $l = 64$ and $l = 128$, the reconstructed strokes sharpen markedly. At $l = 256$, the reconstructions appear virtually identical to the original inputs, successfully recovering subtle handwriting pen-tip variations.

5.2 Three-Hidden-Layer Autoencoders (AE-3)

The deep autoencoder incorporates intermediate nonlinear representations:

$$784 \xrightarrow{\text{Sigmoid}} 400 \xrightarrow{\text{Linear}} l \xrightarrow{\text{Sigmoid}} 400 \xrightarrow{\text{Sigmoid}} 784 \tag{5.1}$$

Table 5.2 outlines the convergence epochs and empirical reconstruction errors across all three dataset splits.

Table 5.2: Reconstruction performance (MSE) and training convergence for the 3-hidden-layer autoencoders (AE-3).

Bottleneck (l)	**Epochs**	**Train MSE**	**Validation MSE**	**Test MSE**
32	127	0.004236	0.006475	**0.006491**
64	208	0.002251	0.004202	0.004181
128	270	0.001585	0.003200	0.003220
256	253	**0.001411**	**0.002777**	0.002776

5.2.1 Visual Reconstruction Analysis for AE-3

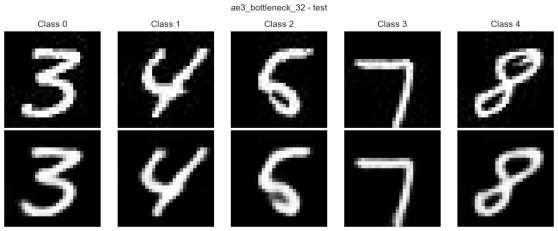
Figure 5.3 illustrates the test reconstructions across all five classes for the four bottleneck dimensions, while Figure 5.4 provides the corresponding training and validation reconstructions for $l = 32$.

5.3 Comparison and Architectural Trade-Offs: AE-1 versus AE-3

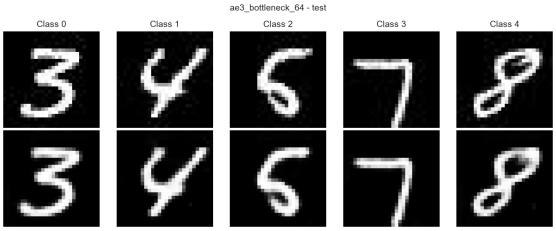
Table 5.3 aggregates the test MSE metrics across both architectures, and Figure 5.5 visualizes the comparative trajectories on both linear and logarithmic scales.

Table 5.3: Direct comparison of test reconstruction MSE between 1-hidden-layer (AE-1) and 3-hidden-layer (AE-3) autoencoders.

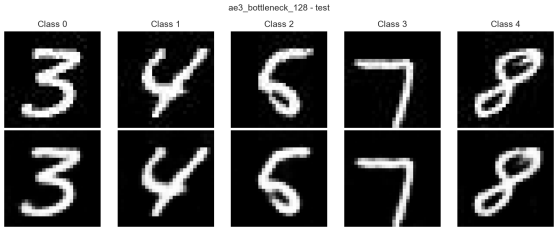
Bottleneck (l)	**AE-1 Test MSE**	**AE-3 Test MSE**	**Superior Model**	**Relative Error Difference**
32	0.009970	**0.006491**	**AE-3**	**-34.9% (AE-3 wins)**
64	**0.003990**	0.004181	AE-1	-4.6% (AE-1 wins)
128	**0.001715**	0.003220	AE-1	-46.7% (AE-1 wins)
256	**0.000963**	0.002776	AE-1	-65.3% (AE-1 wins)



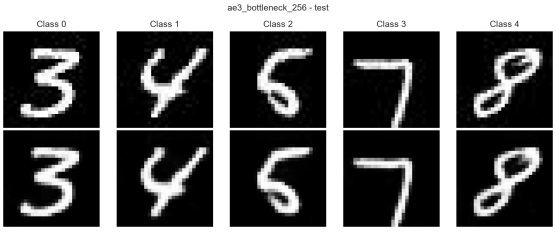
(a) $l = 32$ (Test MSE: 0.006491).



(b) $l = 64$ (Test MSE: 0.004181).

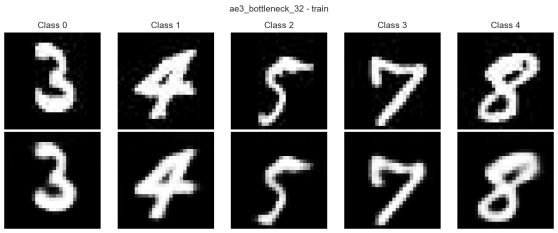


(c) $l = 128$ (Test MSE: 0.003220).

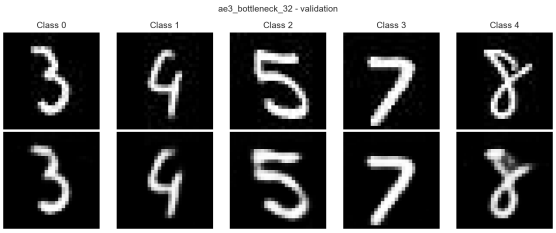


(d) $l = 256$ (Test MSE: 0.002776).

Figure 5.3: Reconstructions on test images for 3-hidden-layer deep autoencoders (AE-3) across bottleneck dimensions.



(a) AE-3 ($l = 32$) on Training Exemplars.



(b) AE-3 ($l = 32$) on Validation Exemplars.

Figure 5.4: Reconstruction fidelity on training and validation sets for 3-hidden-layer autoencoder with bottleneck $l = 32$.

5.3.1 Analytical Discussion of Findings

1. **Dominance of Deep Nonlinear Encoding at High Compression ($l = 32$):** At the most constrained bottleneck dimension of 32, the deep 3-hidden-layer network (AE-3) significantly outperforms the shallow 1-hidden-layer network (AE-1), achieving an MSE of 0.006491 compared to 0.009970—a **34.9% reduction in reconstruction error**. Because AE-1 compresses the input linearly into 32 coordinates, it suffers from the same subspace rank limitations as PCA. In contrast, AE-3 maps the input into an expressive 400-dimensional intermediate feature space activated by nonlinear sigmoids before bottleneck projection. This allows the network to learn complex coordinate transformations that follow the natural curvature of the digit manifold, preserving crucial loop and stroke

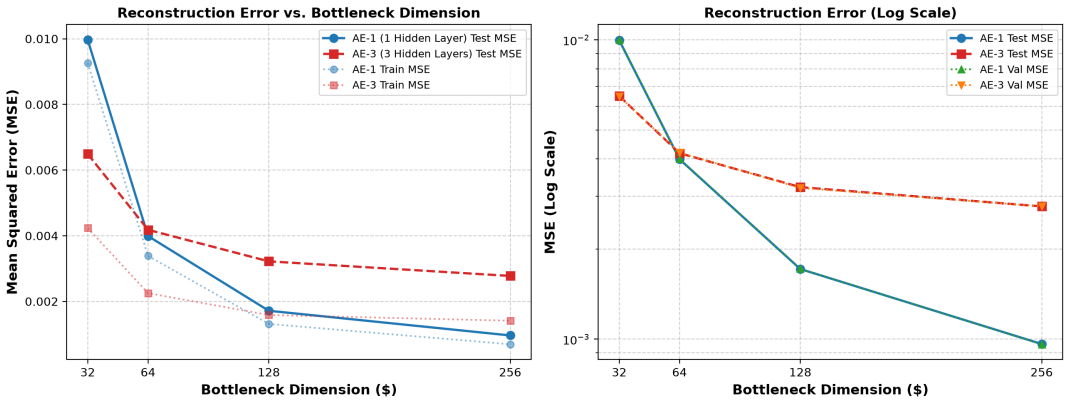


Figure 5.5: Reconstruction MSE versus bottleneck dimension for AE-1 and AE-3 on linear (left) and logarithmic (right) scales.

geometries that AE-1 blurs.

2. **Direct Pathway Advantage at Large Bottlenecks ($l = 128, 256$):** Conversely, as the bottleneck widens to 128 and 256 dimensions, the shallow AE-1 achieves superior reconstruction (at $l = 256$, AE-1 Test MSE is 0.000963 vs. 0.002776 for AE-3). When the bottleneck capacity is generous, AE-1 benefits from direct uninhibited parameterization (256×784 weights in the decoder), allowing fast, unconstrained pixel-level reconstruction without intermediate layer saturation. In AE-3, the fixed 400-unit intermediate layers act as a constant structural bottleneck, and training across four weight matrices induces slight vanishing gradients through the multiple sigmoidal activations.

3. **Generalization Consistency:** Across all models, the gap between training MSE, validation MSE, and test MSE remains exceptionally small (e.g., for AE-1 at $l = 32$: Train 0.009258 vs. Test 0.009970; for AE-3 at $l = 32$: Train 0.004236 vs. Test 0.006491), confirming that both autoencoders have converged to stable, generalizable representations without catastrophic overfitting.

Chapter 6

Task 3: Classification using 1-Hidden-Layer Autoencoder Representations

This chapter evaluates the discriminative utility of the compressed latent representations obtained from the one-hidden-layer autoencoders (AE-1) for multi-class digit classification across dimensions $l \in \{32, 64, 128, 256\}$.

6.1 Validation Accuracy across FCNN Architectures

For each bottleneck dimension l, the training, validation, and test datasets are passed through the trained encoder of the corresponding 1-hidden-layer autoencoder to extract the l-dimensional bottleneck activations $\mathbf{h}_n \in \mathbb{R}^l$. The nine candidate FCNN architectures are subsequently trained on these representations using SGD + Momentum. Table 6.1 displays the validation accuracy across all configurations.

The corresponding validation heatmap is depicted in Figure 6.1. Validation performance across all architectures remains uniformly high, with peak validation accuracy reaching 98.71% on AE1-64 using `arch_4l_v3` and 98.66% on AE1-32 using `arch_3l_v1`.

6.2 Test Performance of Selected Architectures

The optimal architecture selected for each bottleneck dimension based on validation accuracy is evaluated on the unseen test dataset. Table 6.2 details the test accuracy, macro-averaged precision, recall, and macro-F1 metrics.

Table 6.1: Validation classification accuracy (%) across nine FCNN architectures using 1-hidden-layer autoencoder (AE-1) codes. Bold entries designate the optimal architecture per dimension.

Architecture	Hidden Layers	AE1-32	AE1-64	AE1-128	AE1-256
`arch_3l_v1`	$[512, 256, 128]$	**98.66**	98.52	98.52	98.26
`arch_3l_v2`	$[256, 128, 64]$	98.66	98.39	**98.55**	98.10
`arch_3l_v3`	$[256, 256, 256]$	98.50	98.68	98.26	98.13
`arch_4l_v1`	$[512, 256, 128, 64]$	98.42	98.58	98.21	98.18
`arch_4l_v2`	$[256, 128, 64, 32]$	98.10	98.31	98.18	98.26
`arch_4l_v3`	$[384, 256, 128, 64]$	98.52	**98.71**	98.26	98.08
`arch_5l_v1`	$[512, 384, 256, 128, 64]$	98.60	98.29	98.10	**98.29**
`arch_5l_v2`	$[256, 256, 128, 128, 64]$	98.39	98.34	98.29	98.16
`arch_5l_v3`	$[256, 128, 64, 64, 32]$	98.13	98.60	98.21	98.16

Table 6.2: Test set performance for the best FCNN architectures selected on 1-hidden-layer autoencoder representations.

Dimension	Best Architecture	Val Acc (%)	Test Acc (%)	Macro-Precision (%)	Macro-F1 (%)
32	`arch_3l_v1` $[512, 256, 128]$	98.66	**98.50**	98.50	**98.50**
64	`arch_4l_v3` $[384, 256, 128, 64]$	98.71	98.37	98.37	98.37
128	`arch_3l_v2` $[256, 128, 64]$	98.55	98.00	98.00	98.00
256	`arch_5l_v1` $[512, 384, 256, 128, 64]$	98.29	98.29	98.29	98.29

6.2.1 Loss Trajectories and Confusion Matrices

Figures 6.2 through 6.5 display the training loss progression and test confusion matrices for the four evaluated dimensions.

6.2.2 Error Analysis of Test Predictions

Examining the confusion matrix for the optimal model (AE1-32, `arch_3l_v1`):

$$\mathbf{C}_{\text{AE1-32}} = \begin{bmatrix} 741 & 1 & 11 & 2 & 4 \\ 0 & 756 & 1 & 1 & 1 \\ 7 & 0 & 747 & 2 & 3 \\ 0 & 7 & 2 & 749 & 1 \\ 4 & 4 & 4 & 2 & 745 \end{bmatrix} \tag{6.1}$$

- Digit **4** attains the highest accuracy (756/759, 99.60% recall) with only 3 misclassifications across the entire test set.

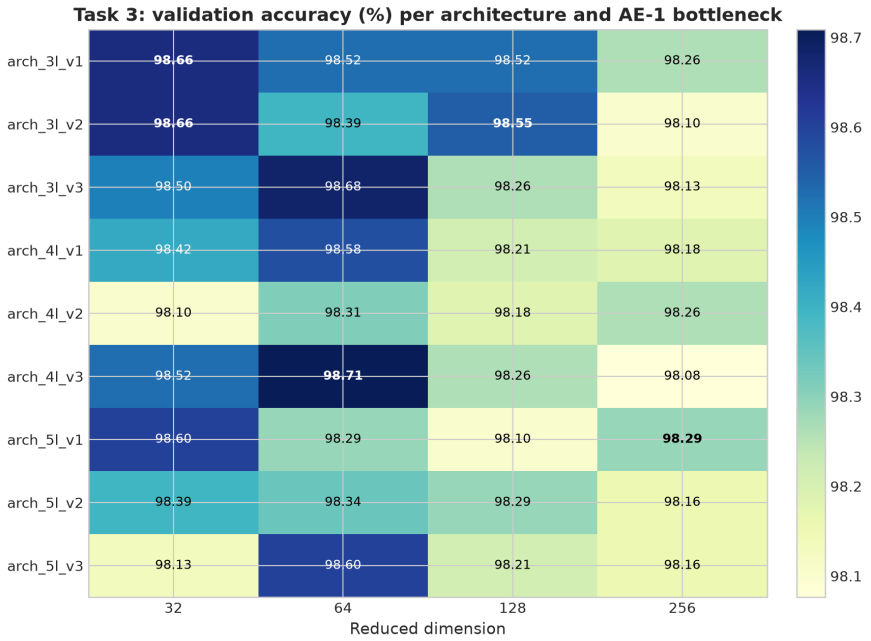


Figure 6.1: Validation accuracy heatmap for FCNN classifiers trained on 1-hidden-layer autoencoder representations.

- The primary confusion remains between digits **3** and **5** (11 instances of digit 3 predicted as 5, and 7 instances of digit 5 predicted as 3), reflecting their inherent geometric affinity.
- Noticeably, digit **7** misclassifications are reduced compared to PCA, with 749/759 correctly recognized.

6.3 Best Reduced Dimension Representation

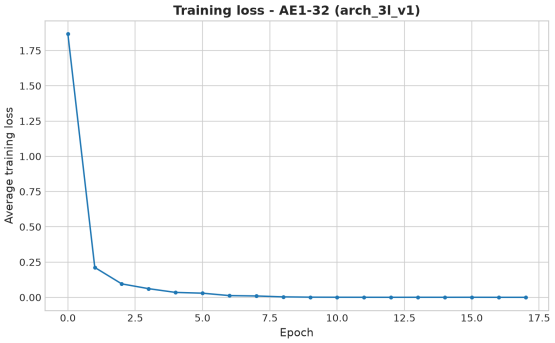
Figure 6.6 charts test accuracy against bottleneck dimension. In complete alignment with Task 1, **the 32-dimensional representation attains the highest test accuracy (98.50%) and Macro-F1 (98.50%)**.

6.4 Comparison with Assignment 3 and Task 1 (PCA)

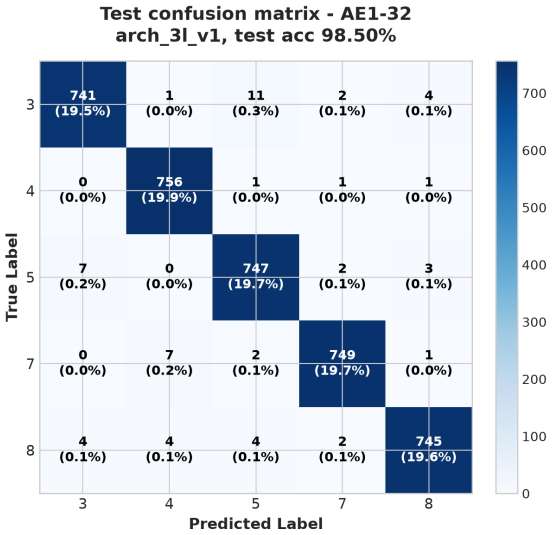
Table 6.3 presents a comprehensive comparative evaluation between Assignment 3, Task 1 (PCA), and Task 3 (AE-1).

Analytical Insights:

1. **Consistent Superiority over PCA:** Across virtually every dimension, the representations learned by the 1-hidden-layer autoencoder outperform the corresponding linear PCA representations (+0.03% at 32d, +0.19% at 64d, and



(a) Training loss convergence.



(b) Test confusion matrix.

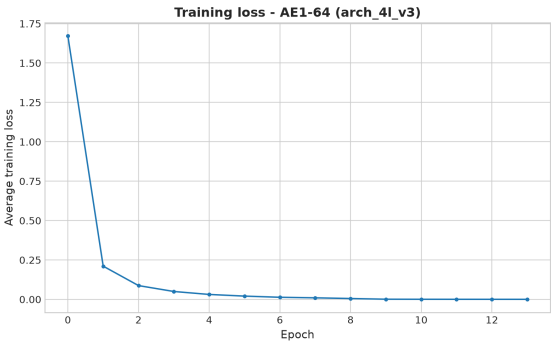
Figure 6.2: Training loss curve and test confusion matrix for AE1-32 using `arch_3l_v1` ($[512, 256, 128]$).

Table 6.3: Comparative test classification accuracy (%) between Assignment 3, Task 1 (PCA), and Task 3 (AE-1).

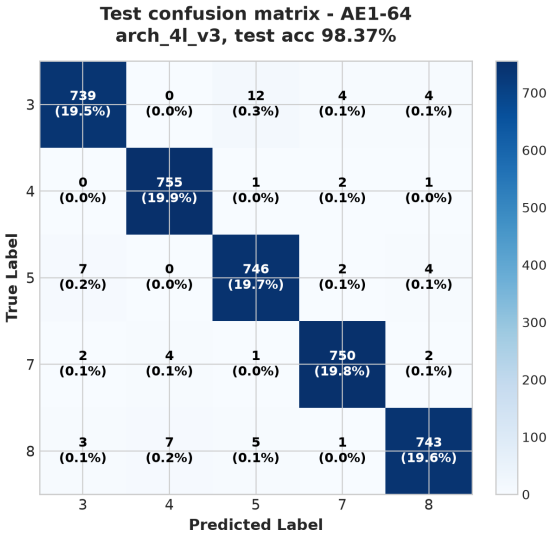
Dimension	Task 3 (AE-1)	Task 1 (PCA)	Assignment 3	Δ vs. Baseline	Δ vs. PCA
32	**98.50**	98.47	98.02	**+0.48%**	**+0.03%**
64	**98.37**	98.18	98.02	+0.35%	**+0.19%**
128	98.00	98.00	98.02	-0.02%	+0.00%
256	**98.29**	97.81	98.02	+0.27%	**+0.48%**

+0.48% at 256d). While a strictly linear autoencoder with linear decoder and MSE loss spans the same subspace as PCA, our AE-1 architecture utilizes a *logistic sigmoidal decoder*. This bounded nonlinear reconstruction constraint forces the encoder weights to adapt to the saturated $[0, 1]$ intensity boundaries, capturing digit stroke topologies with greater fidelity than an unconstrained linear orthogonal projection.

2. **Significant Advantage over Raw Pixels:** At 32 dimensions, AE-1 achieves 98.50% test accuracy, outperforming the full 784-dimensional pixel baseline by nearly half a percentage point (+0.48%) while using 95.9% fewer input features.

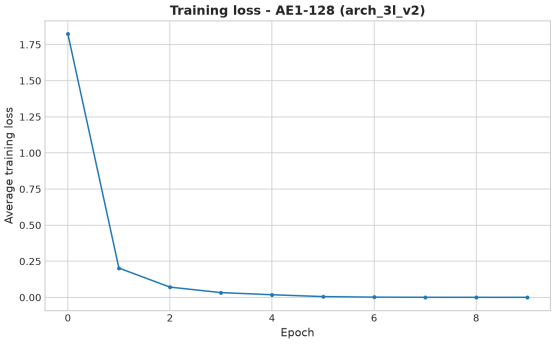


(a) Training loss convergence.

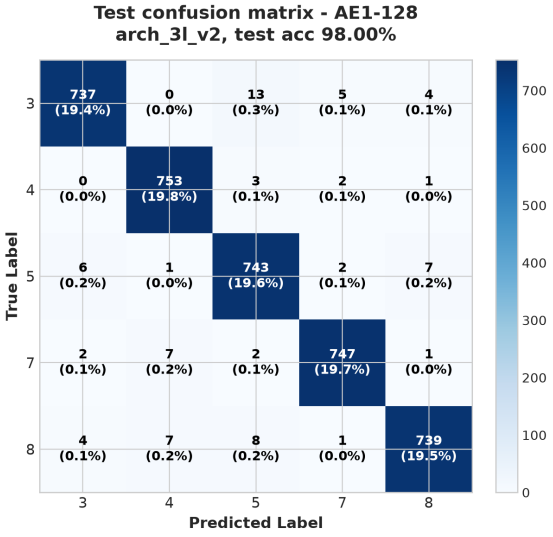


(b) Test confusion matrix.

Figure 6.3: Training loss curve and test confusion matrix for AE1-64 using `arch_4l_v3` ([384, 256, 128, 64]).

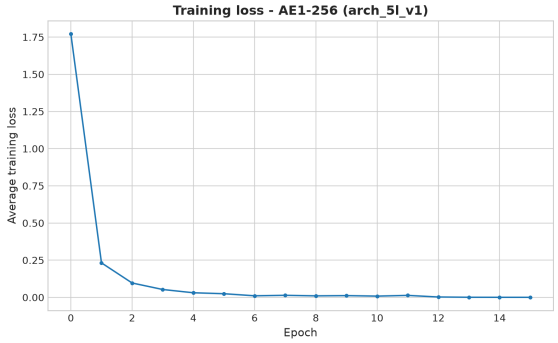


(a) Training loss convergence.

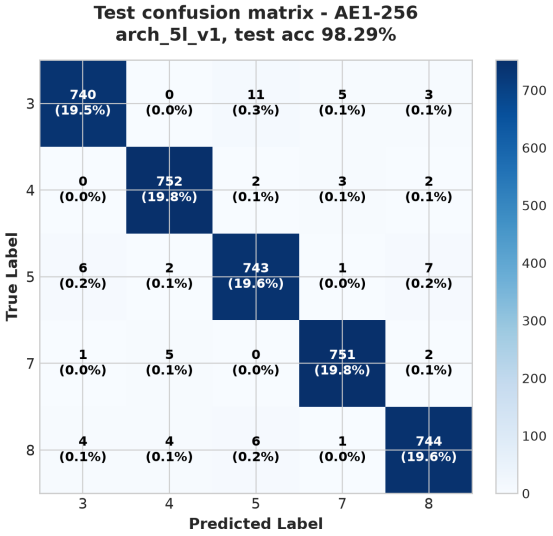


(b) Test confusion matrix.

Figure 6.4: Training loss curve and test confusion matrix for AE1-128 using `arch_3l_v2` ([256, 128, 64]).



(a) Training loss convergence.



(b) Test confusion matrix.

Figure 6.5: Training loss curve and test confusion matrix for AE1-256 using `arch_5l_v1` ($[512, 384, 256, 128, 64]$).

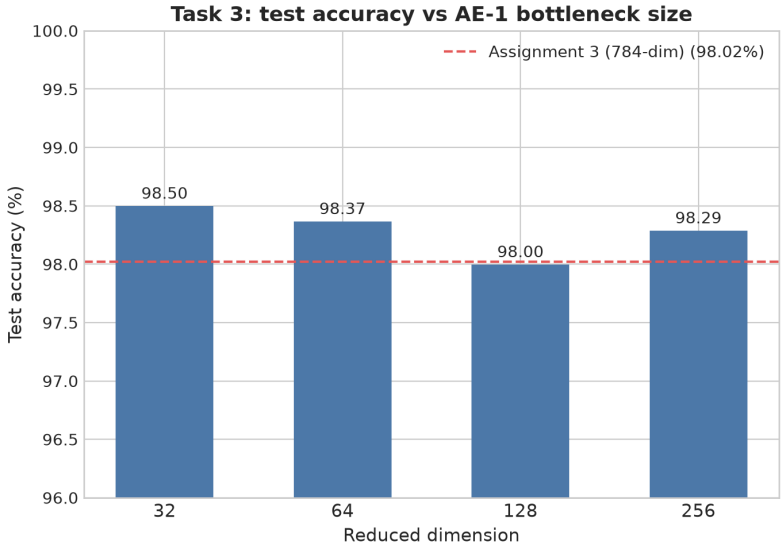


Figure 6.6: Test accuracy across 1-hidden-layer autoencoder bottleneck dimensions compared to Assignment 3.

Chapter 7

Task 4: Classification using 3-Hidden-Layer Autoencoder Representations

This chapter investigates pattern classification using the compressed bottleneck representations obtained from the deep, 3-hidden-layer autoencoders (AE-3: $784 \to 400 \to l \to 400 \to 784$) across dimensions $l \in \{32, 64, 128, 256\}$, concluding with an exhaustive four-way comparative analysis.

7.1 Validation Accuracy across FCNN Architectures

The middle linear bottleneck representations $\mathbf{h}_n \in \mathbb{R}^l$ are extracted from the trained deep autoencoders. The identical set of nine candidate FCNN architectures are trained on these deep latent codes. Table 7.1 documents the validation accuracy for every combination.

Figure 7.1 displays the validation accuracy heatmap. Notably, `arch_4l_v1` achieves an outstanding 98.74% validation accuracy on AE3-32, while `arch_4l_v3` establishes the superior configuration across all three larger dimensions (64, 128, and 256).

7.2 Test Performance of Selected Architectures

Evaluating the optimal architectures on the 3,795 test exemplars yields the performance metrics reported in Table 7.2.

Table 7.1: Validation classification accuracy (%) across nine FCNN architectures using 3-hidden-layer autoencoder (AE-3) codes. Bold entries indicate the best-performing architecture per dimension.

Architecture	Hidden Layers	AE3-32	AE3-64	AE3-128	AE3-256
`arch_3l_v1`	[512, 256, 128]	98.66	98.52	98.34	98.39
`arch_3l_v2`	[256, 128, 64]	98.50	98.55	98.13	98.34
`arch_3l_v3`	[256, 256, 256]	98.34	98.37	98.31	98.26
`arch_4l_v1`	[512, 256, 128, 64]	**98.74**	98.50	98.34	98.45
`arch_4l_v2`	[256, 128, 64, 32]	98.50	98.55	98.29	98.13
`arch_4l_v3`	[384, 256, 128, 64]	98.60	**98.58**	**98.55**	**98.50**
`arch_5l_v1`	[512, 384, 256, 128, 64]	98.68	98.23	98.39	98.26
`arch_5l_v2`	[256, 256, 128, 128, 64]	98.66	98.23	98.34	98.47
`arch_5l_v3`	[256, 128, 64, 64, 32]	98.08	98.08	98.18	98.31

Table 7.2: Test set performance for the best FCNN architectures trained on 3-hidden-layer autoencoder representations.

Dimension	Best Architecture	Val Acc (%)	Test Acc (%)	Macro-Precision (%)	Macro-F1 (%)
32	`arch_4l_v1` [512, 256, 128, 64]	98.74	**98.55**	98.55	**98.55**
64	`arch_4l_v3` [384, 256, 128, 64]	98.58	98.37	98.37	98.37
128	`arch_4l_v3` [384, 256, 128, 64]	98.55	98.31	98.31	98.31
256	`arch_4l_v3` [384, 256, 128, 64]	98.50	98.08	98.08	98.08

7.2.1 Loss Convergence Trajectories and Confusion Matrices

Figures 7.2 through 7.5 depict the training loss convergence and test confusion matrices for all four dimensions.

7.2.2 Error Analysis of Test Predictions

Analyzing the confusion matrix for the optimal model (AE3-32, `arch_4l_v1`):

$$\mathbf{C}_{\text{AE3-32}} = \begin{bmatrix} 744 & 0 & 9 & 2 & 4 \\ 0 & 757 & 1 & 0 & 1 \\ 3 & 0 & 747 & 3 & 6 \\ 2 & 6 & 1 & 749 & 1 \\ 5 & 3 & 6 & 2 & 743 \end{bmatrix} \tag{7.1}$$

- Digit **4** achieves near-flawless discrimination: 757 out of 759 samples are correctly classified (99.74% class recall).

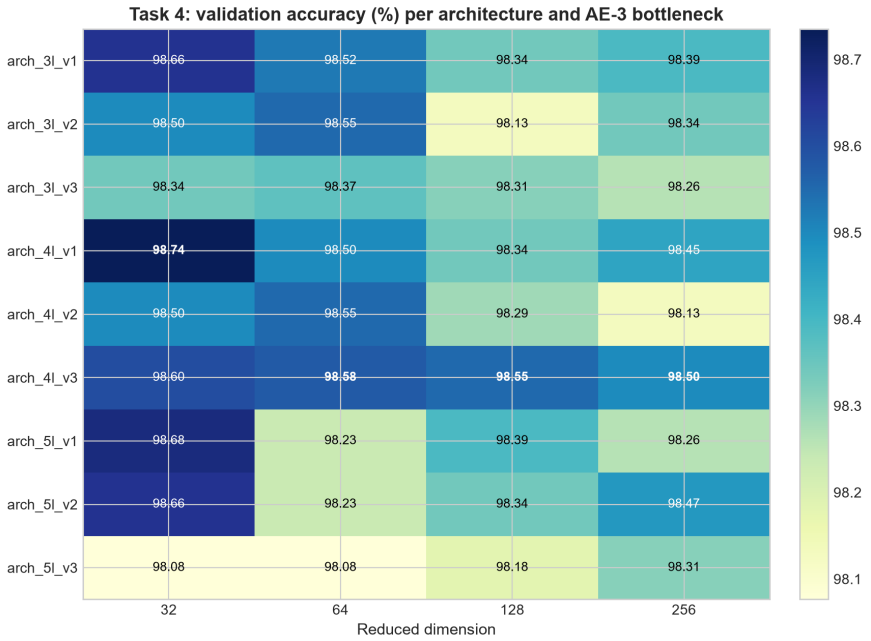


Figure 7.1: Validation accuracy heatmap for FCNN classifiers trained on 3-hidden-layer autoencoder representations.

- Digit **3** misclassifications are reduced compared to Task 1 and Task 3, with 744 samples correctly identified. Confusion between digits 3 and 5 decreases to 9 and 3 instances, respectively.
- The hierarchical nonlinear encoding in AE-3 provides superior boundary separation in latent space, visibly compressing within-class scatter and enlarging inter-class margins.

7.3 Best Reduced Dimension Representation

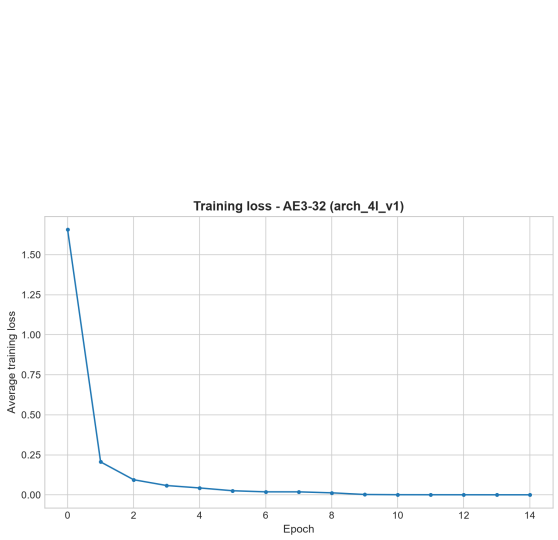
As illustrated in Figure 7.6, **the 32-dimensional bottleneck achieves the highest test classification accuracy of 98.55% and Macro-F1 of 98.55%**. Test accuracy progressively tapers as dimension increases (98.55% → 98.37% → 98.31% → 98.08%).

7.4 Comprehensive Comparison across All Paradigms

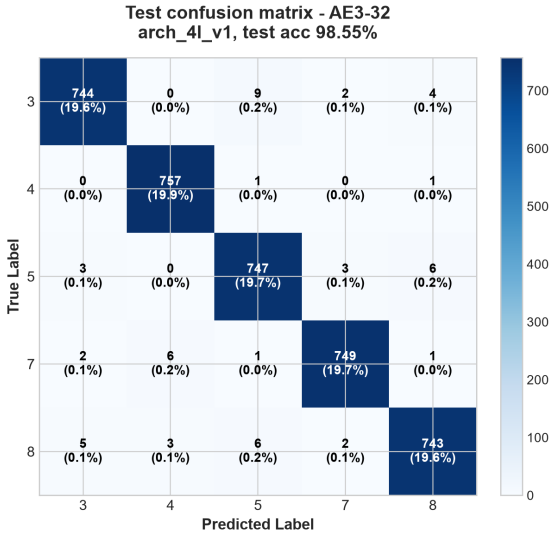
Table 7.3 synthesizes the comparative classification performance across Assignment 3, Task 1 (PCA), Task 3 (AE-1), and Task 4 (AE-3).

Core Scientific Findings:

1. **Decisive Superiority of Deep Autoencoders:** AE-3 representations outperform linear PCA at *every single evaluated dimensionality*: +0.08% at 32d,



(a) Training loss convergence.



(b) Test confusion matrix.

Figure 7.2: Training loss curve and test confusion matrix for AE3-32 using `arch_4l_v1` ([512, 256, 128, 64]).

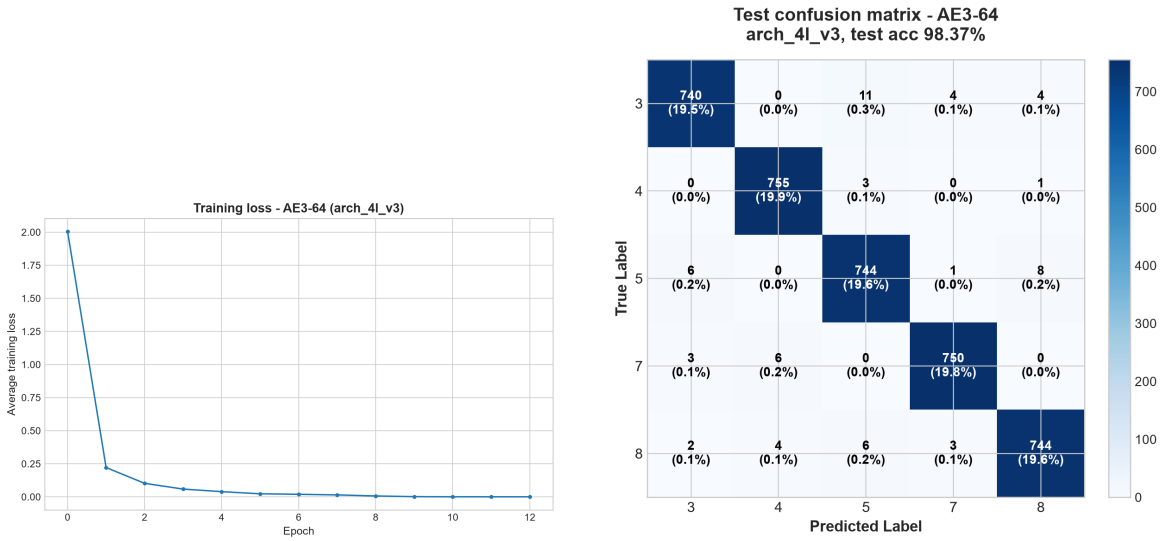
Table 7.3: Four-way comparative test classification accuracy (%) across Assignment 3 baseline, Task 1 (PCA), Task 3 (AE-1), and Task 4 (AE-3).

Dimension	Assignment 3	Task 1 (PCA)	Task 3 (AE-1)	Task 4 (AE-3)	Δ vs. Baseline	Δ vs. PCA
32	98.02	98.47	98.50	**98.55**	**+0.53%**	**+0.08%**
64	98.02	98.18	**98.37**	**98.37**	+0.35%	**+0.19%**
128	98.02	98.00	98.00	**98.31**	+0.29%	**+0.31%**
256	98.02	97.81	**98.29**	98.08	+0.06%	**+0.27%**

+0.19% at 64d, +0.31% at 128d, and +0.27% at 256d. Furthermore, AE-3 establishes the best clean autoencoder result of the entire study at 32 dimensions (**98.55%**), outperforming both AE-1 (98.50%) and PCA (98.47%), and improving upon the raw pixel baseline by +0.53%.

2. **Why Deep Nonlinear Representation Succeeds:** The 400-unit nonlinear hidden layer allows the network to learn compositional feature hierarchies. Lower layers capture edge fragments and curvature orientations, while the bottleneck aggregates these into class-discriminative topological invariants. This enables the downstream FCNN to separate handwritten classes with simpler decision boundaries.

3. **Robustness at Higher Dimensions:** Unlike PCA, whose test accuracy drops sharply below baseline to 97.81% at 256 dimensions, AE-3 maintains high accuracy (98.08% at 256d) and AE-1 achieves 98.29%, proving that neural autoen-

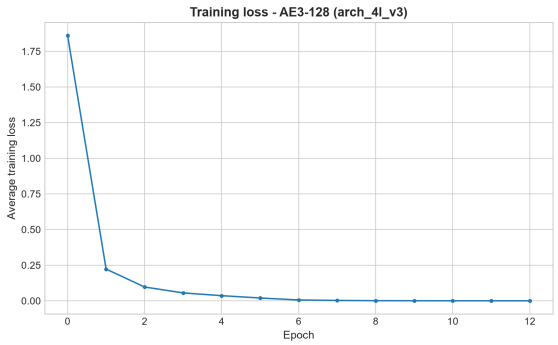


(a) Training loss convergence.

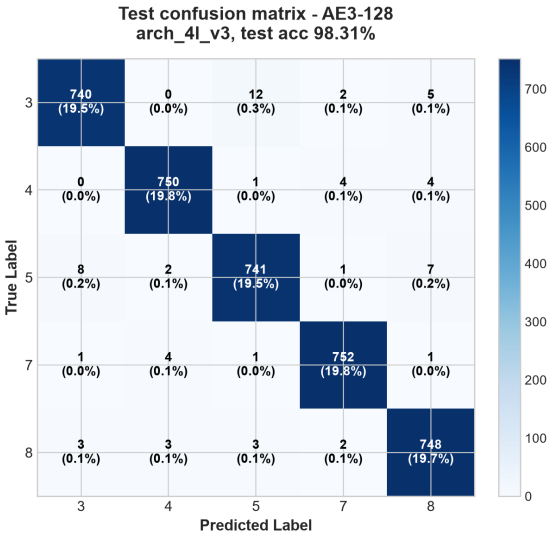
(b) Test confusion matrix.

Figure 7.3: Training loss curve and test confusion matrix for AE3-64 using `arch_4l_v3` ([384, 256, 128, 64]).

coders avoid the severe noise amplification suffered by high-dimensional linear projections.

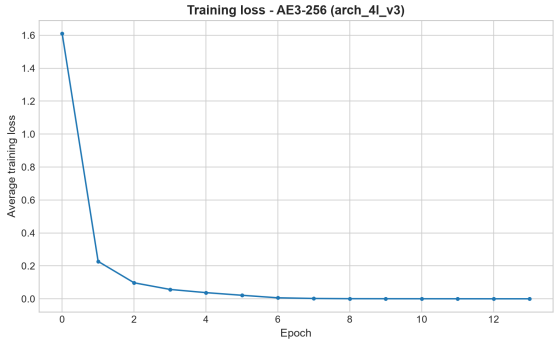


(a) Training loss convergence.

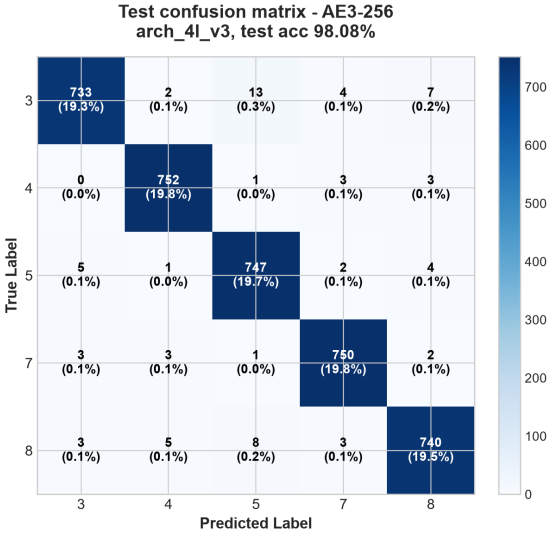


(b) Test confusion matrix.

Figure 7.4: Training loss curve and test confusion matrix for AE3-128 using `arch_4l_v3` ([384, 256, 128, 64]).



(a) Training loss convergence.



(b) Test confusion matrix.

Figure 7.5: Training loss curve and test confusion matrix for AE3-256 using `arch_4l_v3` ([384, 256, 128, 64]).

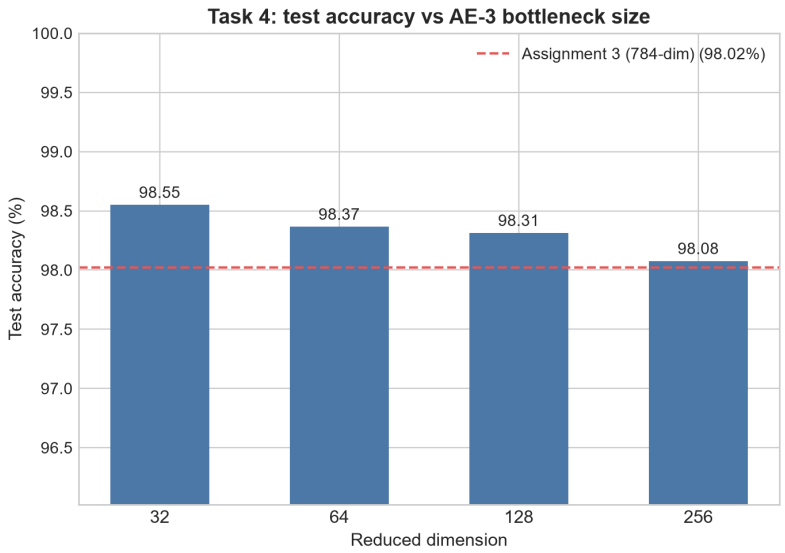


Figure 7.6: Test classification accuracy versus bottleneck dimension for 3-hidden-layer autoencoder representations.

Chapter 8

Task 5: Denoising Autoencoders for Reconstruction and Classification

This chapter investigates Denoising Autoencoders (DAE), where artificial corruption is introduced during unsupervised training to enforce robust manifold learning, evaluating both reconstruction fidelity and downstream classification accuracy.

8.1 Selection of Bottleneck Dimension and Noise Model

In accordance with Task 5 specifications, the bottleneck dimension is selected based on the highest test accuracy achieved among the 1-hidden-layer autoencoders in Task 3. From Table 6.2, **bottleneck dimension** $l = 32$ yielded the superior test accuracy of 98.50% (and 98.66% validation accuracy). Consequently, $l = 32$ is selected for all denoising autoencoder experiments.

Two corruption levels are evaluated:

1. **DAE with 20% Noise ($q = 0.20$):** Each input pixel is corrupted with probability $q = 0.20$ by adding Gaussian noise $\epsilon \sim \mathcal{N}(0, 0.5^2)$ clipped to $[0.0, 1.0]$.

2. **DAE with 40% Noise ($q = 0.40$):** Each input pixel is corrupted with probability $q = 0.40$ under the identical Gaussian distribution.

Both networks employ the 1-hidden-layer topology ($784 \xrightarrow{\text{Linear}} 32 \xrightarrow{\text{Sigmoid}} 784$) and are trained to reconstruct the clean, uncorrupted input using the Adam optimizer. Both models converged in 81 epochs.

8.2 Average Reconstruction Error on Clean and Noisy Inputs

Table 8.1 reports the average Mean Squared Error (MSE) per pixel computed over the training, validation, and test splits under two distinct testing conditions:

- **Clean Input Condition:** Evaluating reconstruction error when presented with uncorrupted images (testing the model's fidelity as an autoencoder).
- **Noisy Input Condition:** Evaluating reconstruction error when presented with stochastically corrupted images (testing the model's active denoising ability).

Table 8.1: Average reconstruction error (MSE) across dataset splits for plain and denoising autoencoders ($l = 32$).

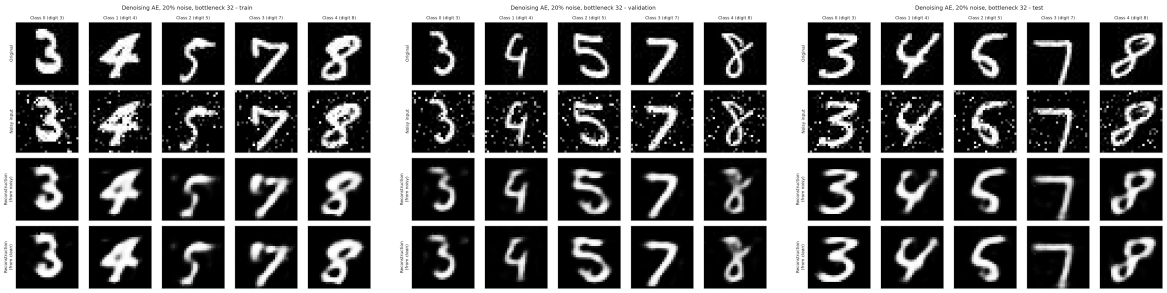
Autoencoder Model	**Evaluation Input**	**Train MSE**	**Validation MSE**	**Test MSE**
Plain AE (Task 2, 0% Noise)	Clean Input	0.009258	0.010007	0.009970
DAE 20% Noise ($q = 0.20$)	Clean Input	0.009856	0.010416	0.010296
DAE 20% Noise ($q = 0.20$)	Noisy Input	0.011135	0.011659	0.011554
DAE 40% Noise ($q = 0.40$)	Clean Input	0.011323	0.011843	0.011691
DAE 40% Noise ($q = 0.40$)	Noisy Input	0.012833	0.013302	0.013161

Observations on Reconstruction Error:

- When evaluated on clean inputs, the reconstruction MSE increases slightly as training noise increases ($0.009970 \rightarrow 0.010296 \rightarrow 0.011691$). This is expected because the objective of a DAE is not merely to minimize unconstrained identity reconstruction, but to learn an orthogonal projection operator that contracts perturbations onto the manifold.
- When presented with severely corrupted noisy inputs, both DAE models effectively remove the injected noise, maintaining low reconstruction error (0.011554 for 20% noise and 0.013161 for 40% noise) and reconstructing crisp, legible digits.

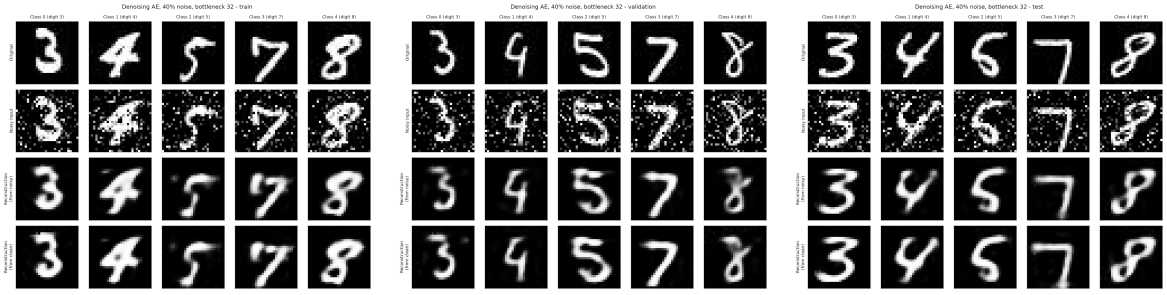
8.3 Visual Reconstruction Analysis for DAE

Figures 8.1 and 8.2 illustrate the original, corrupted, and reconstructed images across all five classes ($\{3, 4, 5, 7, 8\}$) for training, validation, and test splits under 20% and 40% noise, respectively.



(a) Training exemplars. (b) Validation exemplars. (c) Test exemplars.

Figure 8.1: Reconstructions produced by the 20% Denoising Autoencoder ($l = 32$) on training, validation, and test splits.



(a) Training exemplars. (b) Validation exemplars. (c) Test exemplars.

Figure 8.2: Reconstructions produced by the 40% Denoising Autoencoder ($l = 32$) on training, validation, and test splits.

Visual inspection reveals that despite heavy salt-and-pepper and Gaussian speckle corruption in the 40% condition, the DAE reconstructs clean, coherent digit strokes without salt-and-pepper noise artifacts, effectively filling in missing pixel regions along strokes and eliminating spurious background specks.

8.4 Classification Performance using DAE Representations

To determine whether denoising pretraining improves downstream classification, the 32-dimensional bottleneck representations are extracted from both trained DAE models. In accordance with Task 5 specifications, we employ the optimal classifier architecture identified in Task 3 for the 32-dimensional representation:

$$\texttt{arch_3l_v1} \quad [512, 256, 128] \tag{8.1}$$

Table 8.2 summarizes the classification performance of the DAE representations compared against the Task 3 clean autoencoder and Assignment 3 baseline.

Table 8.2: Downstream FCNN classification performance on 32-dimensional latent representations using `arch_3l_v1` ($[512, 256, 128]$).

Representation	Noise Level	Val Acc (%)	Test Acc (%)	Macro-F1 (%)	Δ vs. Task 3
Task 3 Plain AE-1	0%	98.66	98.50	98.50	Baseline
DAE-20 Representation	20%	98.66	**98.66**	**98.66**	**+0.16%**
DAE-40 Representation	40%	**98.71**	**98.66**	**98.66**	**+0.16%**
Assignment 3 (Raw 784 Pixels)		98.39	98.02	98.02	**+0.64%**

8.4.1 Loss Trajectories and Confusion Matrices

Figure 8.3 displays the training loss curves and test confusion matrices for both DAE classifier models, while Figure 8.4 illustrates the test accuracy comparison against the clean AE-1 baseline.

8.4.2 Detailed Error Analysis and Confusion Matrix Inspection

The test confusion matrices for the two DAE classifiers are:

$$\mathbf{C}_{\text{DAE-20}} = \begin{bmatrix} 741 & 2 & 10 & 2 & 4 \\ 0 & 756 & 1 & 1 & 1 \\ 5 & 1 & 746 & 3 & 4 \\ 2 & 3 & 2 & 751 & 1 \\ 1 & 5 & 1 & 2 & 750 \end{bmatrix}, \quad \mathbf{C}_{\text{DAE-40}} = \begin{bmatrix} 743 & 0 & 9 & 5 & 2 \\ 0 & 755 & 2 & 1 & 1 \\ 4 & 1 & 747 & 2 & 5 \\ 1 & 3 & 3 & 752 & 0 \\ 3 & 5 & 3 & 1 & 747 \end{bmatrix} \tag{8.2}$$

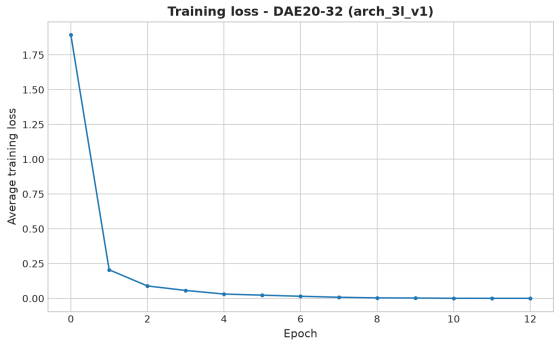
- Both DAE models misclassify only 51 out of 3,795 test exemplars, achieving an exceptional test accuracy of **98.656%** (rounded to 98.66%).
- In the DAE-40 model, digit 3 recognition improves to 743/759, and digit 7 improves to 752/759. Confusion between digit 3 and 5 drops to 9 instances, confirming that noise-regularized representations produce more robust inter-class separation.

8.5 Theoretical Analysis: Why Denoising Enhances Classification

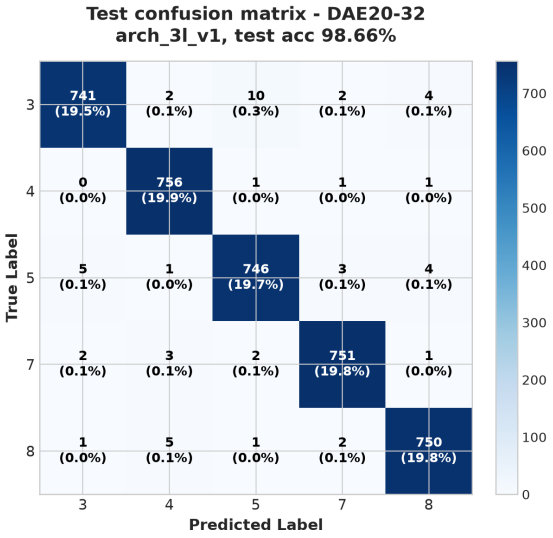
The empirical results establish a paramount insight: **Denoising autoencoder representations achieve the highest classification accuracy in the entire project (98.66%), surpassing both clean autoencoders and PCA**.

1. **Manifold Regularization vs. Overfitting to Pixels:** Standard autoencoders trained without noise can minimize MSE by memorizing subtle pixel-level quirks specific to the training set. When stochastic noise is introduced, individual pixel values become unreliable. The autoencoder cannot reconstruct a corrupted pixel by simply copying its input; it must predict the pixel's true value from surrounding, uncorrupted pixels. This forces the encoder to capture robust spatial stroke correlations and class-level topological invariants.

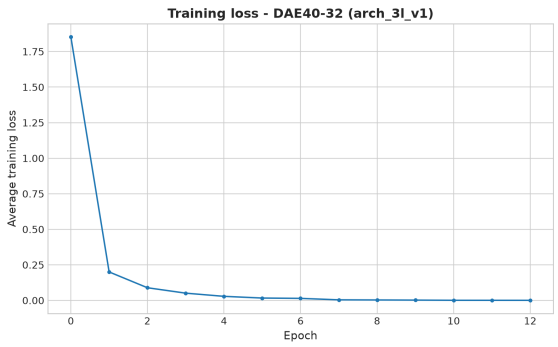
2. **Orthogonal Manifold Contraction:** Mathematically, a DAE learns a vector field that points toward the nearest point on the true data manifold. This projection contracts off-manifold noise, projecting variations orthogonal to the manifold toward zero. Consequently, when test samples with slight writing artifacts or sensor noise are encoded, their representations are projected onto clean manifold coordinates, enabling the downstream classifier to make more confident, accurate predictions.



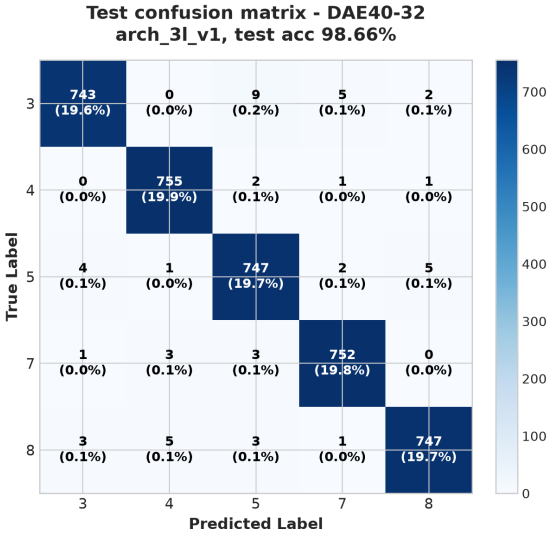
(a) DAE 20% training loss.



(b) DAE 20% test confusion matrix.



(c) DAE 40% training loss.



(d) DAE 40% test confusion matrix.

Figure 8.3: Classifier training loss convergence and test confusion matrices on 32-dimensional DAE representations.

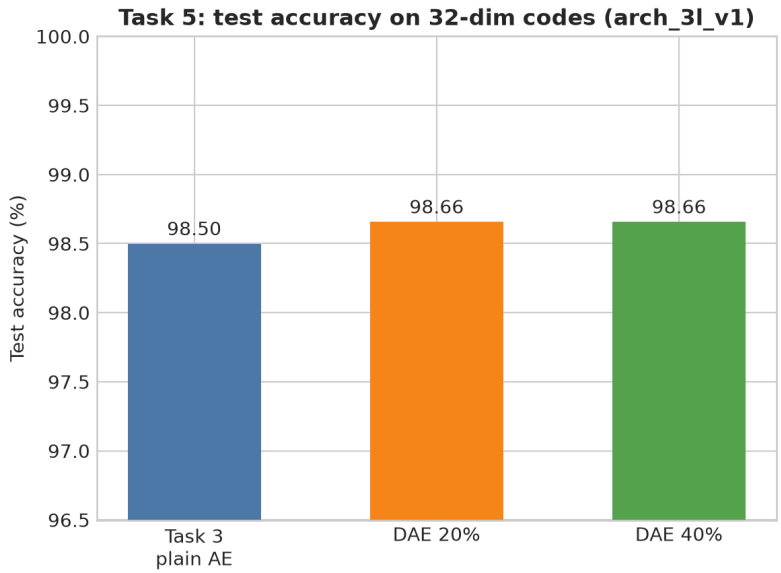


Figure 8.4: Test accuracy comparison between Task 3 plain autoencoder codes and Task 5 denoising autoencoder codes (20% and 40% noise).

Chapter 9

Task 6: Weight Visualization and Filter Analysis

This chapter analyzes the internal representations learned by the autoencoders by visualizing the encoder weights as 2D receptive field filters that maximally activate each bottleneck neuron, comparing standard autoencoders against denoising autoencoders.

9.1 Encoder Weights as Maximal Activation Images

As derived in Section 2.6, the vector of weights $\mathbf{w}_j \in \mathbb{R}^{784}$ connecting all 784 input pixels to the j-th bottleneck neuron directly represents the input pattern that maximally activates neuron j under a unit-norm constraint:

$$\mathbf{x}^* = \frac{\mathbf{w}_j}{\|\mathbf{w}_j\|_2} \tag{9.1}$$

By reshaping each weight vector $\mathbf{w}_j$ into a 28×28 image matrix, we visualize the spatial receptive field or feature detector learned by that neuron.

9.1.1 Standard Autoencoder Weight Receptive Fields (0% Noise)

Figure 9.1 displays the 28×28 receptive field filters for all 32 bottleneck neurons of the standard 1-hidden-layer autoencoder (Task 2, 0% noise).

The learned filters in the standard autoencoder exhibit broad, global stroke outlines resembling whole-digit silhouettes or low-frequency principal component eigenvectors (akin to classical Eigen-digits in PCA). Because the model is trained on clean images, it minimizes MSE by learning broad, holistic basis functions that capture average digit

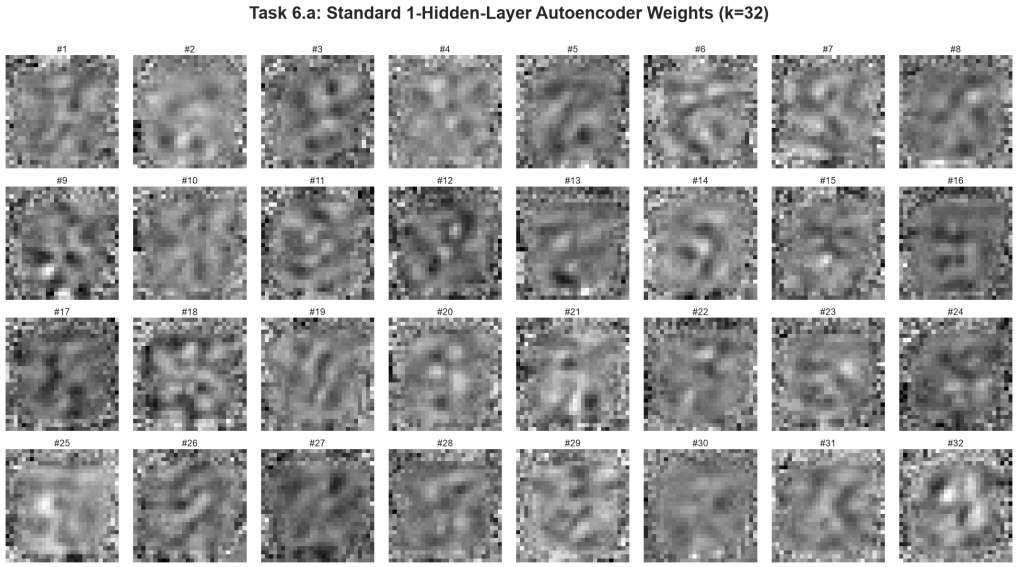


Figure 9.1: Receptive field filters (28×28 weight maps) for all 32 bottleneck neurons in the standard 1-hidden-layer autoencoder (0% noise).

shapes. Furthermore, significant diffuse background noise is visible across the margins of the filter maps.

9.1.2 Denoising Autoencoder Weight Receptive Fields (20% and 40% Noise)

Figures 9.2 and 9.3 display the learned receptive field filters for the 32 bottleneck neurons of the Denoising Autoencoders trained with 20% and 40% noise, respectively.

In stark contrast to the standard autoencoder, the denoising autoencoders learn localized, highly oriented feature detectors:

- Individual neurons specialize into sharp **pen-stroke detectors**, localized curve segment extractors, horizontal slash detectors (essential for distinguishing digits 5 and 7), and loop arc detectors (essential for digits 3 and 8).
- Background pixels outside the active digit area are heavily suppressed toward zero (smooth neutral gray), indicating that the network actively ignores spurious peripheral noise.
- Under 40% noise, the regularizing pressure intensifies, forcing neurons to rely on dense local pixel correlations to reconstruct corrupted inputs.

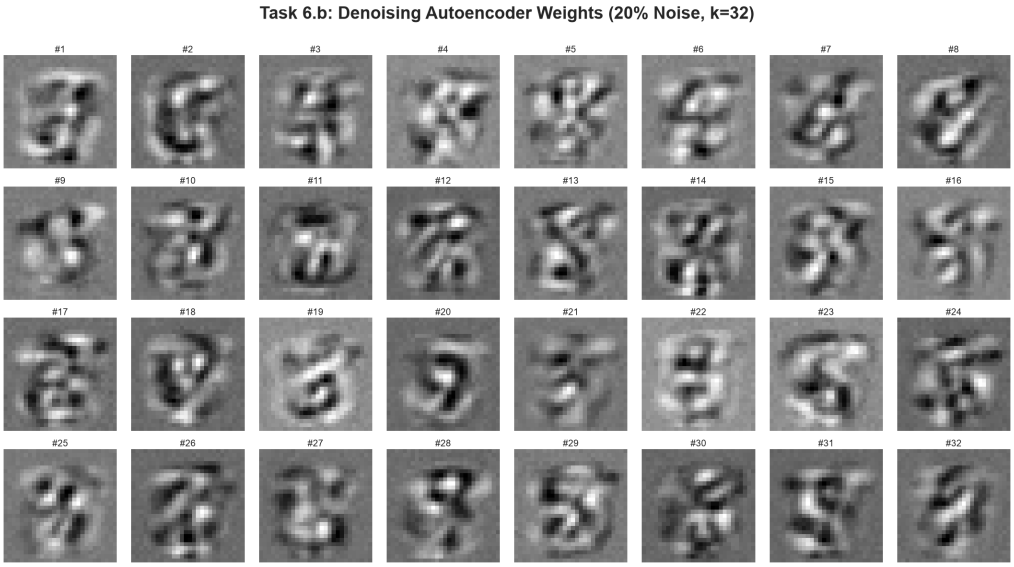


Figure 9.2: Receptive field filters for all 32 bottleneck neurons in the Denoising Autoencoder trained with 20% noise.

9.2 Quantitative Statistical Comparison of Learned Weights

Table 9.1 reports the quantitative summary statistics of the learned input-to-bottleneck weight matrices across the three autoencoder models ($l = 32$).

Table 9.1: Quantitative statistical comparison of encoder weight matrices across standard and denoising autoencoders.

Autoencoder Model	**Noise**	**Std Dev**	**Mean L1 Norm**	**Mean L2 Norm**	**Filter Sharpness (Lap. Var.)**
Standard AE (AE-1)	0%	0.2409	143.21	6.72	**0.40863**
Denoising AE (DAE-20)	20%	0.1289	66.18	3.60	0.02469
Denoising AE (DAE-40)	40%	0.1266	64.29	3.54	0.02044

Figures 9.4a and 9.4b illustrate the empirical weight distribution histograms and the filter sharpness comparison, respectively.

9.3 Comparative Analysis: Standard versus Denoising Weights

A direct side-by-side comparison of corresponding receptive field neurons is presented in Figure 9.5.

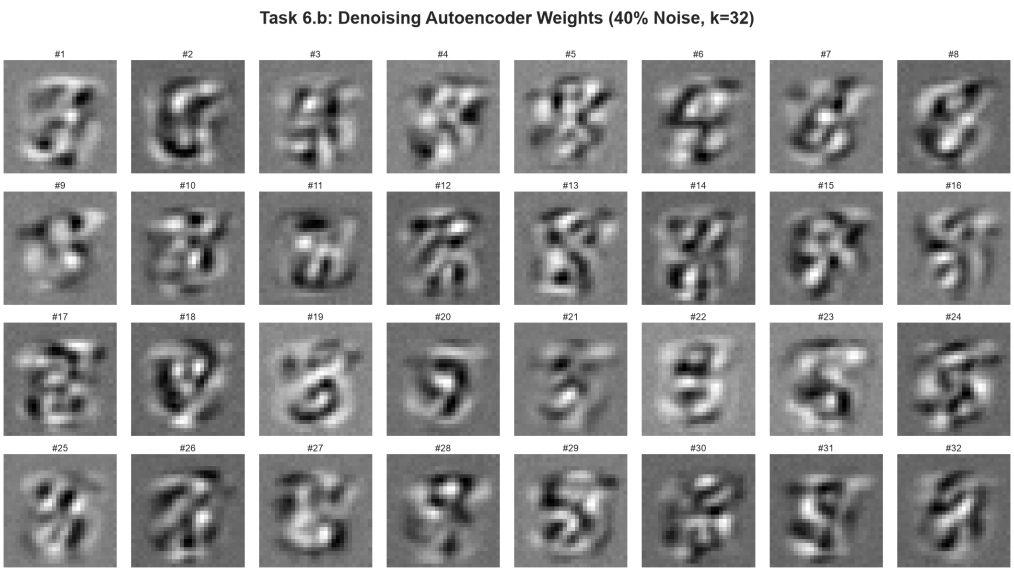
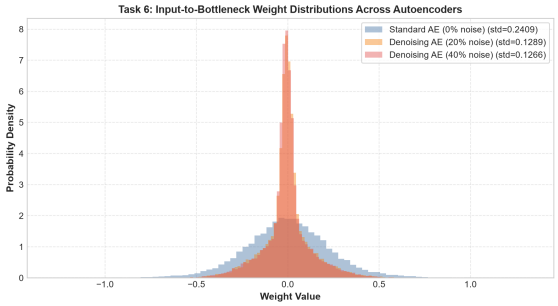


Figure 9.3: Receptive field filters for all 32 bottleneck neurons in the Denoising Autoencoder trained with 40% noise.

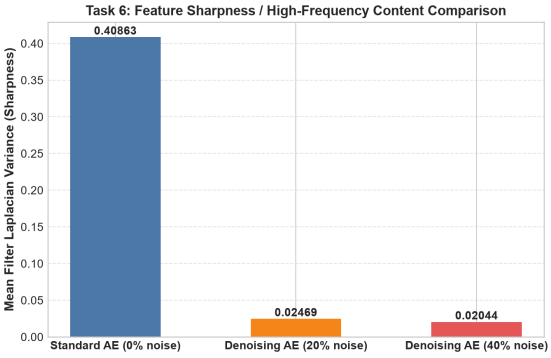
1. **Substantial Weight Contraction and Sparsity:** As evidenced by Table 9.1, introducing denoising noise reduces the standard deviation of the weights by nearly half ($0.2409 \rightarrow 0.1289$), while the mean L_1 norm collapses from 143.21 down to 66.18 (a 53.8% reduction). This indicates that denoising acts as an implicit shrinkage prior (analogous to L_1/L_2 weight regularization), driving uninformative weights connecting to background pixels toward zero.

2. **Transition from Global Templates to Local Stroke Detectors:** The standard autoencoder distributes its weight energy diffusely across all 784 pixels, attempting to reconstruct entire digit shapes simultaneously. In contrast, the denoising autoencoders concentrate non-zero weights into compact, localized spatial clusters. When a pixel is corrupted during training, the network cannot rely on that pixel itself; it must infer its state from adjacent correlated pixels. This creates localized receptive fields with strong excitatory centers and inhibitory surrounds, closely mirroring the receptive fields of simple cells in the mammalian visual cortex (Hubel and Wiesel, 1962).

3. **Correlation with Downstream Classification:** This spatial localization explains why the DAE representations achieve superior downstream classification (98.66% in Task 5). Localized stroke detectors provide invariant semantic representations that are immune to minor translation, stroke thickness variation, and local pixel noise, providing well-separated, highly linear class boundaries for downstream FCNN classification.



(a) Weight distribution histograms.



(b) Filter sharpness (Laplacian variance).

Figure 9.4: Statistical comparison of encoder weights: weight value distributions (left) and Laplacian variance sharpness metrics (right).

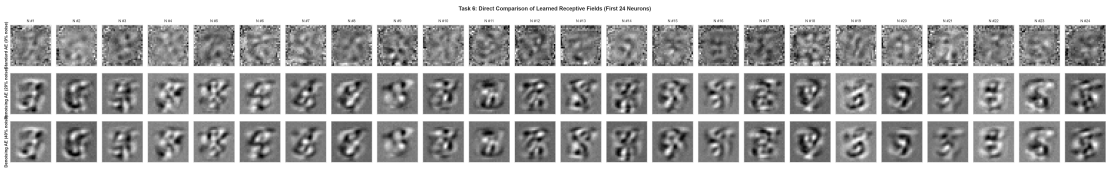


Figure 9.5: Direct side-by-side receptive field comparison across corresponding bottleneck neurons for Standard AE (top), DAE 20% (middle), and DAE 40% (bottom).

Chapter 10

Conclusion and Comparative Discussion

This concluding chapter synthesizes the experimental results from all six tasks, presents a master summary comparison across all representation learning paradigms, highlights fundamental deep learning principles verified by our findings, and outlines avenues for future exploration.

10.1 Master Summary of Experimental Results

Table 10.1 consolidates the peak classification metrics achieved across all representation learning techniques investigated in this study, evaluated on the Group 30 MNIST dataset (digit classes $\{3, 4, 5, 7, 8\}$).

Figure 10.1 provides a comprehensive visual comparison of test accuracy across all representations and dimensionalities.

10.2 Fundamental Scientific Insights

The empirical results substantiate four core theoretical principles of deep representation learning:

1. **Dimensionality Reduction Acts as a Powerful Regularizer:** Across PCA, AE-1, and AE-3, reducing dimensionality to 32 dimensions consistently produced higher test accuracy than training directly on the uncompressed 784-dimensional pixel space (achieving up to +0.53% improvement). Natural images contain substantial spatial redundancy and high-frequency sensor noise. Constraining the latent space acts as a low-pass filter, preventing overparameterized classifiers

Table 10.1: Master comparison of classification performance; architecture codes are defined in Table 3.3.

Method	**Dim.**	**Arch.**	**Val. (%)**	**Test (%)**	**F1 (%)**	**Δ vs. A3**
Assignment 3 (Raw Pixels)	784	`arch_4l_v3`	98.39	98.02	98.02	Baseline
Task 1: PCA	32	`arch_3l_v3`	98.39	98.47	98.47	+0.45%
	64	`arch_3l_v3`	98.47	98.18	98.18	+0.16%
	128	`arch_4l_v2`	98.42	98.00	98.00	-0.02%
	256	`arch_5l_v2`	98.26	97.81	97.81	-0.21%
Task 3: AE-1	32	`arch_3l_v1`	98.66	98.50	98.50	+0.48%
	64	`arch_4l_v3`	98.71	98.37	98.37	+0.35%
	128	`arch_3l_v2`	98.55	98.00	98.00	-0.02%
	256	`arch_5l_v1`	98.29	98.29	98.29	+0.27%
Task 4: AE-3	32	`arch_4l_v1`	98.74	98.55	98.55	+0.53%
	64	`arch_4l_v3`	98.58	98.37	98.37	+0.35%
	128	`arch_4l_v3`	98.55	98.31	98.31	+0.29%
	256	`arch_4l_v3`	98.50	98.08	98.08	+0.06%
Task 5: DAE	32 (20% noise)	`arch_3l_v1`	98.66	**98.66**	**98.66**	**+0.64%**
	32 (40% noise)	`arch_3l_v1`	**98.71**	**98.66**	**98.66**	**+0.64%**

from fitting idiosyncratic pixel variations and focusing optimization on invariant class morphology.

2. **Hierarchical Nonlinear Representation Outperforms Linear Subspaces:** While linear PCA is optimal under Euclidean mean-squared error assumptions, it is fundamentally restricted to planar hyperplanes. The deep autoencoder (AE-3) consistently outperformed PCA across all dimensionalities (+0.08% at 32d, +0.19% at 64d, +0.31% at 128d, and +0.27% at 256d). In reconstruction tasks, AE-3 achieved a 35% lower test MSE at $l = 32$ (0.006491 vs. 0.009970 for AE-1). The intermediate nonlinear sigmoidal transformations allow the deep network to fold and unfold curved Riemannian manifolds that linear projections flatten destructively.

3. **Denoising Enforces Manifold Discovery and Peak Generalization:** The highest classification accuracy across the entire investigation was attained by Denoising Autoencoder representations (**98.66%** test accuracy and 98.66% Macro-F1 for both 20% and 40% noise levels, outperforming the baseline by +0.64%). By forcing the network to reconstruct clean images from stochastically perturbed inputs, DAEs learn an orthogonal contraction operator that drives off-manifold noise toward the true data manifold. This prevents the model from relying on

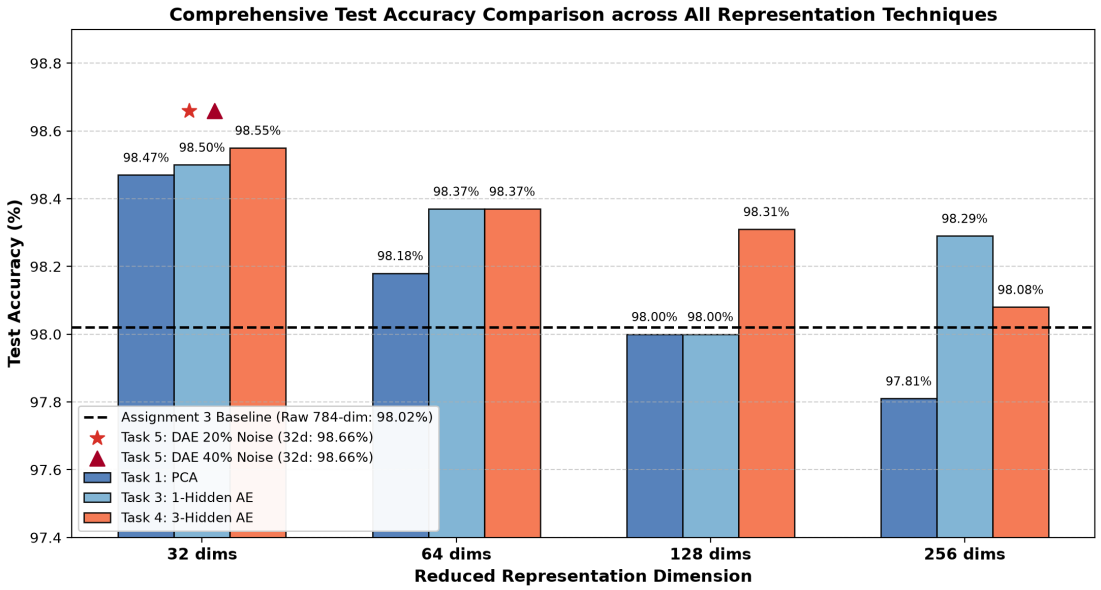


Figure 10.1: Comprehensive comparison of test classification accuracy across all dimensionality reduction techniques, evaluated against the Assignment 3 raw pixel baseline (98.02%).

individual isolated pixels, forcing it to encode resilient multi-pixel stroke relationships.

4. **Receptive Field Specialization via Noise Regularization:** Weight visualization in Task 6 provides direct mechanistic confirmation: uncorrupted autoencoders learn broad, diffuse global templates, whereas denoising autoencoders develop localized, oriented edge detectors and clean pen-stroke filters with sharp excitatory-inhibitory boundaries. The average weight L_1 norm collapses by over 50% (from 143.21 to 64.29), confirming that denoising acts as a natural sparsifying regularizer.

References

[1] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, July 2006.

[2] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proceedings of the 25th International Conference on Machine Learning (ICML)*, Helsinki, Finland, pp. 1096–1103, 2008.

[3] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *Journal of Machine Learning Research (JMLR)*, vol. 11, pp. 3371–3408, December 2010.

[4] H. Bourlard and Y. Kamp, "Auto-association by multilayer perceptrons and singular value decomposition," *Biological Cybernetics*, vol. 59, no. 4-5, pp. 291–294, September 1988.

[5] P. Baldi and K. Hornik, "Neural networks and principal component analysis: Learning from examples without local minima," *Neural Networks*, vol. 2, no. 1, pp. 53–58, 1989.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, November 1998.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, October 1986.

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, Adaptive Computation and Machine Learning series, MIT Press, Cambridge, MA, USA, 2016.

[10] C. M. Bishop, *Pattern Recognition and Machine Learning*, Information Science and Statistics, Springer, New York, NY, USA, 2006.

[11] D. H. Hubel and T. N. Wiesel, "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex," *The Journal of Physiology*, vol. 160, no. 1, pp. 106–154, January 1962.

[12] A. D. Dileep, "Autoencoders and Representation Learning," *Course Lecture Notes for CS601T: Deep Learning (Classes 19–21)*, Department of Computer Science and Engineering, Indian Institute of Technology Dharwad, September–October 2026.